LAB 2775

From Bucket to RAG: Event-Driven Docling for watsonx and Confluent

Hands-on lab guide

Michele Dolfi dol@zurich.ibm.com
Senior Technical Staff Member, IBM Software

Panos Vagenas pva@zurich.ibm.com
Advisory Engineer, IBM Software

Cesar Berrospi Ramis ceb@zurich.ibm.com
Senior Researcher, IBM Software

Peter Staar taa@zurich.ibm.com
Principal RSM, Manager, IBM Software



Notices and disclaimers

© 2026 International Business Machines Corporation. No part of this document may be reproduced or transmitted in any form without written permission from IBM.

U.S. Government Users Restricted Rights — use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM.

This document is current as of the initial date of publication and may be changed by IBM at any time. Not all offerings are available in every country in which IBM operates.

Information in these presentations (including information relating to products that have not yet been announced by IBM) has been reviewed for accuracy as of the date of initial publication and could include unintentional technical or typographical errors. IBM shall have no responsibility to update this information.

This document is distributed “as is” without any warranty, either express or implied. In no event, shall IBM be liable for any damage arising from the use of this information, including but not limited to, loss of data, business interruption, loss of profit or loss of opportunity. IBM products and services are warranted per the terms and conditions of the agreements under which they are provided. The performance data and client examples cited are presented for illustrative purposes only. Actual performance results may vary depending on specific configurations and operating conditions.

IBM products are manufactured from new parts or new and used parts. In some cases, a product may not be new and may have been previously installed. Regardless, our warranty terms apply.

Any statements regarding IBM's future direction, intent or product plans are subject to change or withdrawal without notice.

Performance data contained herein was generally obtained in a controlled, isolated environments. Customer examples are presented as illustrations of how those customers have used IBM products and the results they may have achieved. Actual performance, cost, savings or other results in other operating environments may vary.

References in this document to IBM products, programs, or services does not imply that IBM intends to make such products, programs or services available in all countries in which IBM operates or does business.

Workshops, sessions and associated materials may have been prepared by independent session speakers, and do not necessarily reflect the views of IBM. All materials and discussions are provided for informational purposes only, and are neither intended to, nor shall constitute legal or other guidance or advice to any individual participant or their specific situation.

It is the customer's responsibility to ensure its own compliance with legal requirements and to obtain advice of competent legal counsel as to the identification and interpretation of any relevant laws and regulatory requirements that may affect the customer's business and any actions the customer may need to take to comply with such laws. IBM does not provide legal advice or represent or warrant that its services or products will ensure that the customer follows any law.

Notices and disclaimers (continued)

Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products. IBM does not warrant the quality of any third-party products, or the ability of any such third-party products to interoperate with IBM's products. IBM expressly disclaims all warranties, expressed or implied, including but not limited to, the implied warranties of merchantability and fitness for a purpose.

The provision of the information contained herein is not intended to, and does not, grant any right or license under any IBM patents, copyrights, trademarks or other intellectual property right.

IBM, the IBM logo, and ibm.com are trademarks of International Business Machines Corporation, registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the Web at “Copyright and trademark information” at: www.ibm.com/legal/copytrade.shtml.

Apache, Apache Flink, Apache Kafka and the Flink and Kafka logos are trademarks of the Apache Software Foundation. Confluent is a trademark of Confluent, Inc.

Table of contents

1 Introduction 6
1.1 A two-minute introduction to Docling 6
1.2 A two-minute introduction to event-driven ingestion 7
1.3 What you will build 8
1.4 The five phases 9
1.5 Conventions and acronyms 9
1.6 The documents 10
2 Getting started 12
2.1 Connect to your lab VM 12
2.2 Your student identity, and where the workshop's values are 13
2.3 Set up the lab 14
2.4 The workshop coordinates 17
3 Phase 1 — Your own Docling instance 19
3.1 Start the trial 19
3.2 Convert a document, and look at what comes out 20
3.3 Collect your service URL and API key 21
4 Phase 2 — A bucket that fires events 24
4.1 Sign in to IBM Cloud 24
4.2 Create your bucket 25
4.3 Subscribe your bucket to the trigger app 28
4.4 Do not upload yet 32
5 Phase 3 — See the chunks arrive 33
5.1 Check your configuration 33
5.2 Create your topics 34
5.3 Create your OpenSearch index 35
5.4 Start the inspection dashboard 36
5.5 Upload a document and watch the chunks arrive 38
6 Phase 4 — The simple enrichment pipeline 42
6.1 Deploy it 42
6.2 What the pipeline does 43

6.3 Watch the replay 44
6.4 Read an enriched record 44
6.5 Search what you just indexed 46
6.6 Ask your own corpus a question 47
6.7 Upload another document 49
7 Phase 5 — The full pipeline 51
7.1 Switch pipelines 51
7.2 What is new 52
7.3 Document A — an ordinary document 53
7.4 Document B — a document that contains personal data 54
7.5 The three guards 55
7.6 Document C — the same content, in a different file 56
7.7 Change the rules while it runs 57
7.8 What you just did 58
8 Where to take it next 59
9 Did not finish the lab? 59
10 Getting help 59
10.1 Ask the lab instructor 59
10.2 Common problems 60
10.3 Useful commands 61
11 Reference material 61

1 Introduction

In this lab you build the ingestion half of a RAG system, and you build it the way it is built in production: as a chain of events rather than as a script somebody remembers to run. A document lands in a bucket, and without anyone touching a keyboard again it is converted, chunked, cleaned, de-duplicated, embedded, and written to a stream that any number of consumers can read.

Four products do the work, and each one does exactly one job:

- **Docling for IBM watsonx** turns a document into structured content and then into chunks.
- **IBM Cloud Object Storage** and **IBM Cloud Code Engine** turn an upload into an event.
- **Confluent Platform** carries the chunks, and **Apache Flink** enriches them as they pass.
- **watsonx.ai** turns each chunk into a vector.

The documents come from one company, and so does the problem. **Halvorsen Fluid Systems**, a fictional maker of industrial pumps, wants an assistant that answers its field engineers' questions from the company's own documents: technical notes from engineering, the maintenance manuals, and the case files the service desk keeps for every customer call-out. Those documents arrive in a bucket from different teams on their own schedule. The case files contain customers' e-mail addresses, phone numbers and payment details, which must never reach the index the assistant searches. The same technical note gets filed twice, by two departments. And the rules about what counts as personal data change while the pipeline runs. Each phase of the lab deals with one of those problems, and §1.6 lists the documents you use for each.

You will finish with a running pipeline of your own, on a shared cluster, that you can re-configure while it is running — and with a clear picture of which part of the system each of those products is responsible for.

1.1 A two-minute introduction to Docling

Docling is an open-source document-processing toolkit, started at IBM Research in Zurich and now hosted by the LF AI & Data Foundation. **Docling for IBM watsonx** is the supported, hosted version of it — the one you will use today.

What it does is narrower and deeper than “extract the text”:

- **It reconstructs the document.** Page layout, reading order, headings, lists, tables, formulas, code blocks and figures — recovered as structure, not as a flat string. A two-column paper comes out in reading order; a table comes out as a table.
- **It reads more than PDF.** DOCX, PPTX, XLSX, HTML, Markdown, images and much more. Scanned pages go through OCR.

- **It chunks.** The chunker is token-aware and structure-aware: it splits on the document's own boundaries, keeps a chunk's headings attached to it, and sizes chunks to the token budget of the embedding model that will consume them.
- **It can write straight to Kafka.** The hosted service's output target produces one Kafka message per chunk, so Docling is a stream producer and not just a converter. That single fact is what makes today's architecture possible.

> **NOTE**
> Retrieval quality is bounded by chunk quality. A chunk that starts mid-sentence, or that has been separated from the heading it belongs under, cannot be cited convincingly no matter how good the embedding model is. This is why conversion and chunking are done by a document tool and not by 20 lines of PyPDF.

1.2 A two-minute introduction to event-driven ingestion

The obvious way to ingest documents is a batch job: every hour, list a folder, convert whatever is new. It works until it does not. It has an hour of latency built in, it re-scans everything to find the few new files, a failure halfway through leaves you guessing what was processed, and adding a second consumer means editing the job.

The event-driven version inverts it. Writing the object **is** the event:

- Object Storage emits a notification the moment the object is written.
- Code Engine receives that notification and hands the object to Docling.
- Docling writes one Kafka message per chunk.
- Flink consumes that topic — and so can anything else you add later.

Three properties come out of that shape, and you will see all three today:

- **Each step fails alone.** If Docling is slow, the upload still succeeded and the notification is retried. If your Flink job is not running, the chunks queue on the topic and wait.
- **The topic is a log, not a queue.** Deploying a new pipeline in Phase 4 replays every chunk you uploaded in Phase 3, because the job reads from the beginning. Re-processing history is the default, not a special mode.
- **Delivery is at-least-once, everywhere.** Retries mean duplicates are normal, so the pipeline has a de-duplication stage rather than an assumption. You will demonstrate it deliberately in Phase 5.

1.3 What you will build

What happens when you drop a file in the bucket

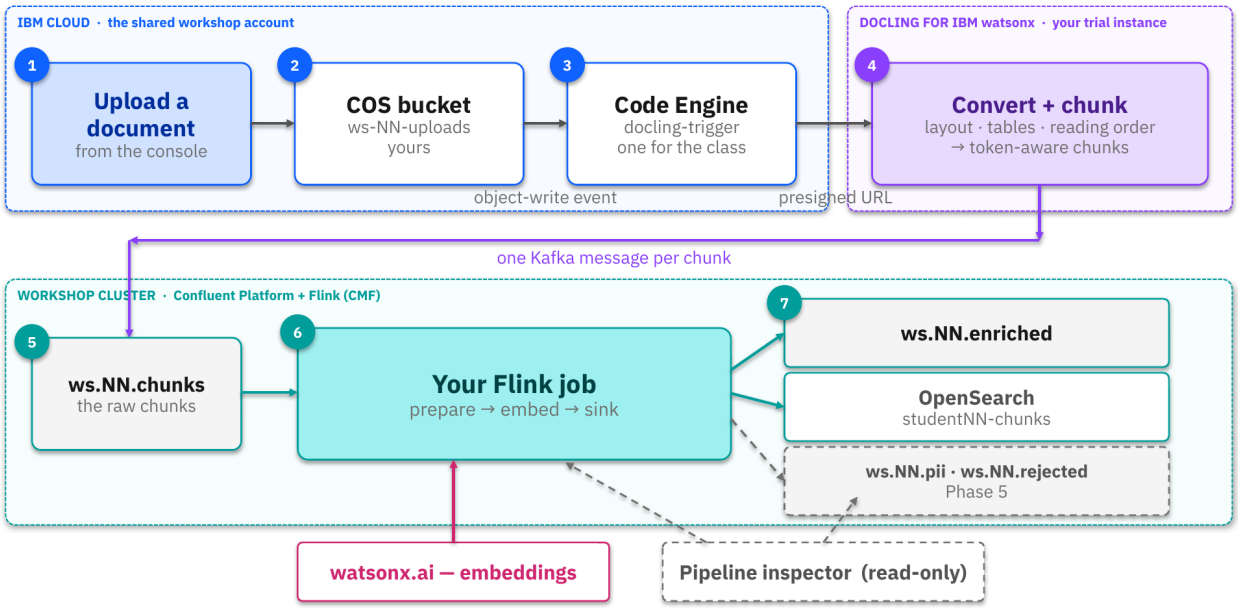


Figure 1. The whole lab on one page. One manual step — the upload; everything after it is an event. You create ② and its subscription in Phase 2, ④ in Phase 1, ⑤ and the inspector in Phase 3, and ⑥ in Phases 4 and 5. ⑦ and the index beside it are what the job writes.

Read it as a single path with one manual step at the far left:

#	What it is	Who set it up
1	You upload a document, from the IBM Cloud console.	you, every time you want a run
2	**Your** COS bucket. The write raises an object-created event.	you, in Phase 2
3	The `docling-trigger` app on Code Engine. It receives the event, signs a one-object URL and submits it to Docling. It holds no state of yours.	the instructor, once for the class
4	**Your** Docling instance: convert, chunk, and produce the chunks to Kafka.	you, in Phase 1
5	`ws.NN.chunks` — your chunk topic on the workshop's Confluent cluster.	you, in Phase 3
6	**Your** Flink job, deployed through Confluent Manager for Apache Flink.	you, in Phases 4 and 5
7	`ws.NN.enriched` — and in Phase 5 two more topics beside it.	created with the rest in Phase 3

> **NOTE**
> Notice what is **not** on the diagram: any code of yours running between the steps. There is no polling loop, no scheduler, and no glue script. The only thing you run by hand all day is the upload.

1.4 The five phases



Figure 2. What each phase leaves you with.

Phase	What you do	You leave with	Min
1	Start a Docling for IBM watsonx trial and convert a document in its workbench.	a service URL and an API key	10
2	Create your own COS bucket and subscribe it to the trigger app.	an upload that starts a pipeline run	15
3	Point the inspection dashboard at your topics, then upload a document.	chunks on a Kafka topic	15
4	Deploy the simple enrichment pipeline on Flink.	enriched, embedded chunks	15
5	Switch to the full pipeline: policy broadcast, PII guard, side outputs.	a pipeline you can re-rule without redeploying	20

The lab is 90 minutes and the activities are sized for 75, which leaves room for the parts of Phase 1 and 2 that depend on somebody else's cloud. **Every phase works on its own** — if you fall behind, ask the instructor for the values the earlier phase would have produced and carry on from there.

1.5 Conventions and acronyms

- Your **student id** is a two-digit number, which the bootstrap prints for you in §2.3. Wherever you see `ws.NN.chunks` or `ws-NN-simple`, substitute yours: student 6 uses `ws.06.chunks` and `ws-06-simple`, which is also the id in this guide's screenshots. Write it here: _____
- Grey boxes are commands you type in a terminal on the lab VM. Lines starting with `#` are comments, not commands.

- Text in `this typeface` is something you type or something the system shows you literally.

Acronym	Expansion	In this lab
RAG	Retrieval-Augmented Generation	why you are chunking and embedding at all
COS	(IBM) Cloud Object Storage	the bucket you upload to
CE	(IBM Cloud) Code Engine	runs the small app that reacts to the upload event
CMF	Confluent Manager for Apache Flink	the control plane your Flink job is deployed to
CRN	Cloud Resource Name	the identifier you copy for your bucket
PII	Personally Identifiable Information	what the Phase 5 guard detects and redacts
TTL	Time To Live	how long the de-duplication stage remembers a chunk
DAG	Directed Acyclic Graph	the shape of a Flink job — the boxes in the dashboard

1.6 The documents

Four PDFs come with the lab, in `lab-documents/`. Everything in them is invented: the company, the people, the card number and the bank account. They are laid out like real company documents, with tables whose headers are grouped over two rows and, in the manual, text in two columns, so recovering the table structure and the reading order is real work for Docling. Each one is there to make one thing happen, so upload them in the order the guide asks for.

File	What it is	Where	What it shows
`01-technical-note.pdf`	TN-2026-014, *Bearing temperature drift on the HX-400 pump series*: headings, a figure, two tables, part numbers	Phases 3 to 5	the ordinary case: chunked, enriched, indexed, and clean of personal data
`04-maintenance-manual.pdf`	OM-HX4, the series' operation and maintenance manual: specifications, schedules, fault finding, spare parts	Phase 4	a second document to retrieve from, which the technical note partly overrides
`02-customer-record.pdf`	service case SC-88213, a bearing failure at a brewery: contacts, site details, billing, a refund	Phase 5	e-mail addresses, a card number and an IBAN redacted by default; phone numbers and an IP address once you enable those detectors

File	What it is	Where	What it shows
`03-technical-note-reissued.pdf`	the same note as 01, filed again: a plain typed layout, US Letter instead of A4, a different typeface and title	Phase 5	a different file with the same words, which de-duplication collapses

When to upload: look for the purple blocks

You never have to work out yourself when to upload a document. Wherever the guide wants one, it shows a block like this, with the file's name in large type:



EXAMPLE · DO NOT UPLOAD YET

`lab-documents/NN-the-file-the-guide-names.pdf`

Which phase this is, and where to upload it: always your bucket, in the **private** window.

In the guide itself the label says **UPLOAD NOW**. There are six of these blocks in the guide, and nothing else in it asks you to upload. For the best experience, upload exactly when you reach one and not before: each phase shows something about what is *already* in your index, so a document that arrives early either spoils the demonstration it was meant for or ends up somewhere it should not be. The dashboard's **Upload a document** panel lists the same order, by phase, if you lose your place.

WATCH OUT

Do not upload `02-customer-record.pdf` before Phase 5. The simple pipeline in Phase 4 has no PII guard, so the card number and the IBAN would go into your index in clear.

2 Getting started

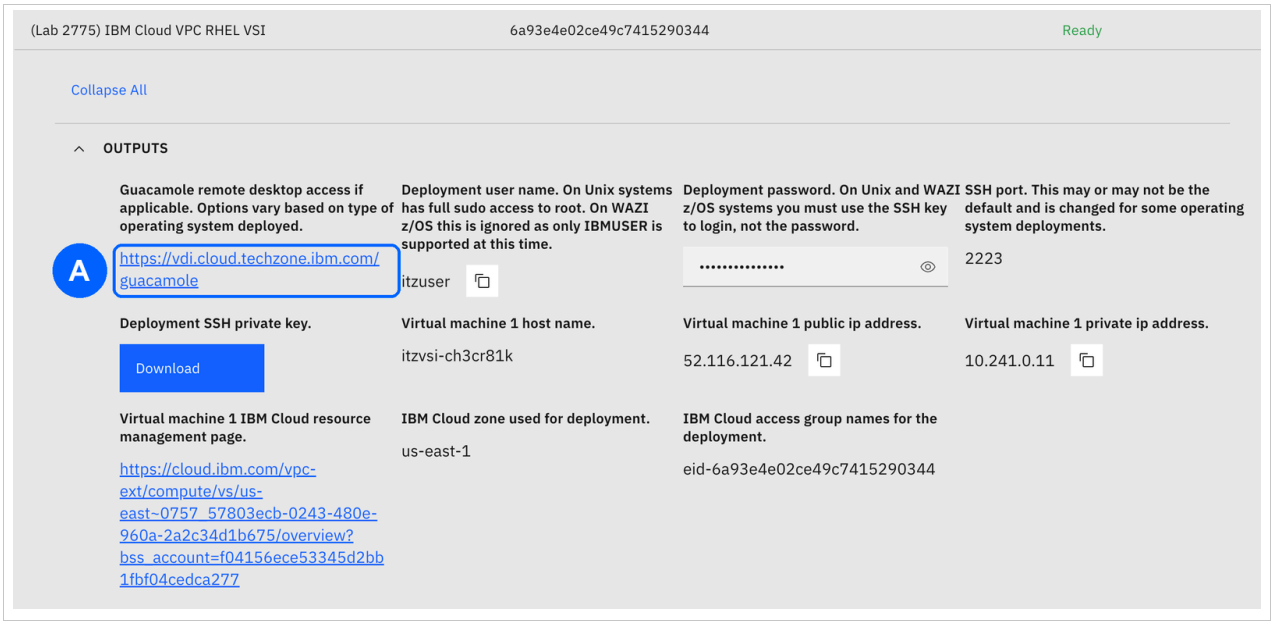
Ten minutes of setup that every later phase depends on. Do all of it before Phase 1.

2.1 Connect to your lab VM

Everything you run in this lab runs on a Red Hat Enterprise Linux virtual machine with a desktop, which you reach through your browser. It has Firefox, a terminal and the tools already installed.

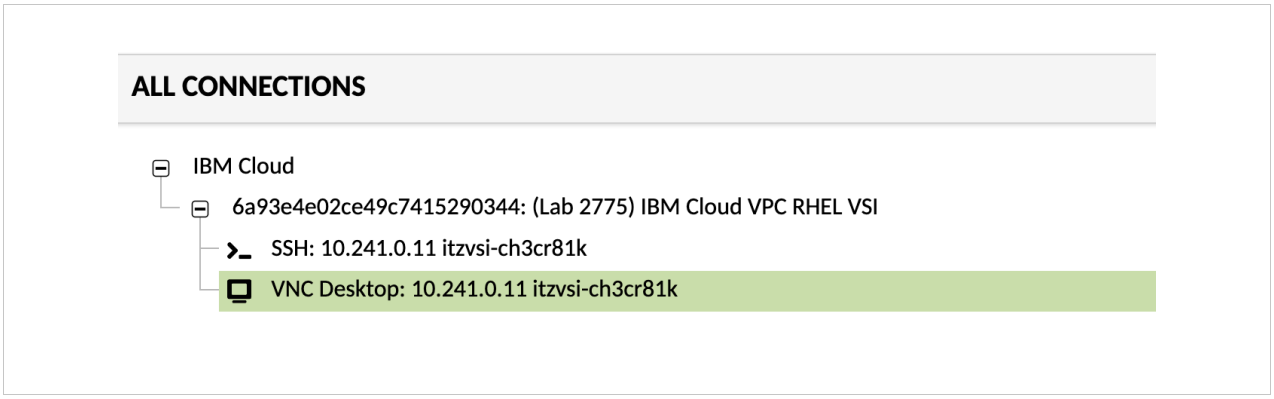
Your lab details page — the one you land on after registering — is where the VM is. It is the only thing you need from that page. Everything else, including your IBM Cloud sign-in for Phase 2, is already on the VM.

1. Expand its **Outputs** and open the **Guacamole remote desktop access** link (A).

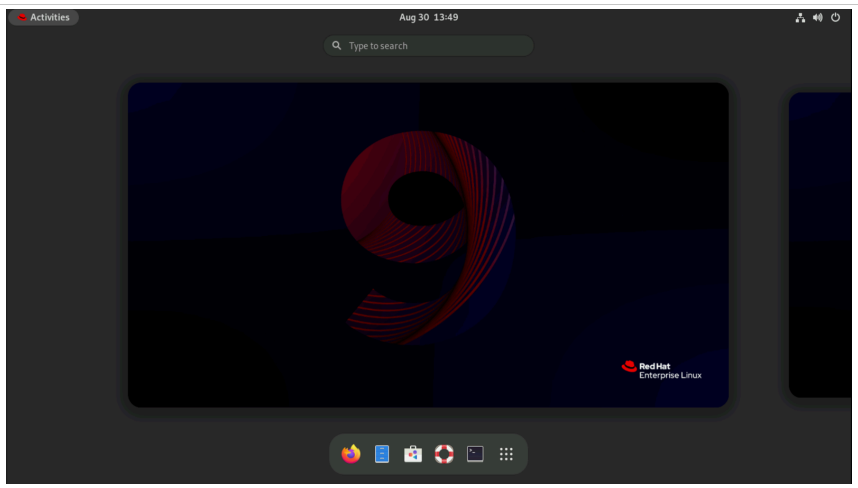


Screen 1. The VM environment's **Outputs** panel. The remote-desktop link (A) is the only thing on it this lab needs.

2. Under **All connections**, choose the **VNC Desktop** entry — not the SSH one. SSH gives you a shell without a browser, and Phases 1 and 2 happen in a browser.
3. You should get a desktop with Firefox and a terminal application. Leave the browser open — you will use it for the IBM Cloud console and for the Docling workbench.



Screen 2. Two connections are offered for the same machine. Take the highlighted one.



Screen 3. The desktop you land on. Firefox and the terminal are both in the dock at the bottom.

2.2 Your student identity, and where the workshop's values are

Three things are yours for the day:

What	Where it comes from	Used for
Your **student id**	already on your VM — the bootstrap in §2.3 prints it	names your topics (`ws.06.chunks`), your bucket and your Flink application
Your **workshop values**	already on your VM — the bootstrap in §2.3 sets them up	the Kafka and Flink credentials, your OpenSearch password, and the watsonx.ai project
Your **IBM Cloud sign-in** — a link, a username and a password	already on your VM — the page `ibmcloud-signin.html` in your home folder	signing in to the workshop's IBM Cloud account in Phase 2

All three arrive together. Before the lab, the instructors put two files in your VM's home folder, written for the student who sits at it. One is for the lab's scripts: your id, your own OpenSearch password, and the addresses and credentials of the shared systems your pipeline runs on. You never open it. The bootstrap in §2.3 sets it up, and from then on the scripts read it for you. The other, `ibmcloud-signin.html`, is for you, and you open it in Phase 2.

Two different logins

Today you sign in to IBM Cloud twice, as two different people. Keep them apart:

	Who you sign in as	Where the login comes from	Which window
Phase 1 — your Docling trial	**you**: your own IBMid, your personal account	you create it, or already have one	an ordinary Firefox window
Phase 2 — the IBM Cloud console	**a workshop student**: a username that is not an IBMid	`ibmcloud-signin.html`, already on your VM	a **private** window
Phase 3 on — the lab dashboard	nobody: it runs on your VM	`./pipeline.sh inspect`	an ordinary window, but its link to your bucket goes in the **private** one

WATCH OUT

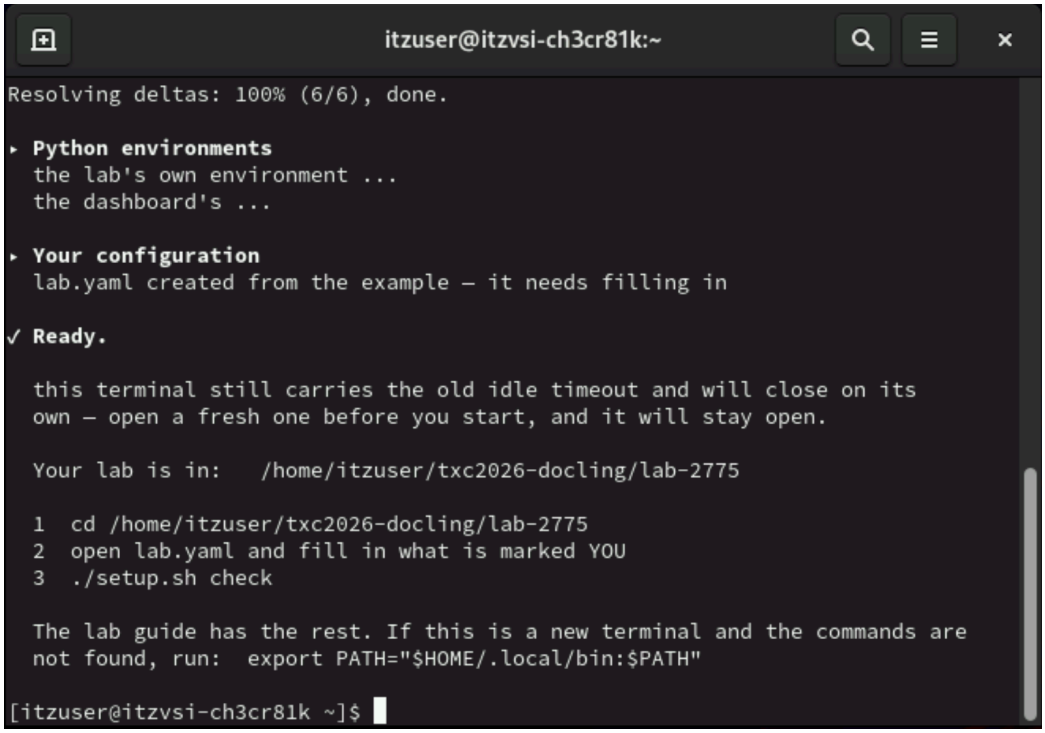
Both logins happen on `cloud.ibm.com`, so in one window the second replaces the first. Your IBMid does not work for the workshop login, and the workshop login does not own your Docling trial. Mixing them up is the most common way to lose ten minutes today, and it can also put your bucket in the wrong account. Keep Phase 1 in ordinary windows and Phase 2 in a private one.

2.3 Set up the lab

Open a terminal on the VM and run one command:

```
curl -L ibm.biz/txc26-2775-bootstrap | bash -
```

It fetches the lab, installs the tooling it needs and builds the two Python environments — a Kafka client and the tools the lab scripts use, and separately the ones the dashboard needs. Neither installs Flink: your Flink job runs on the cluster, in a container image that is already built and published. It takes a minute or two.

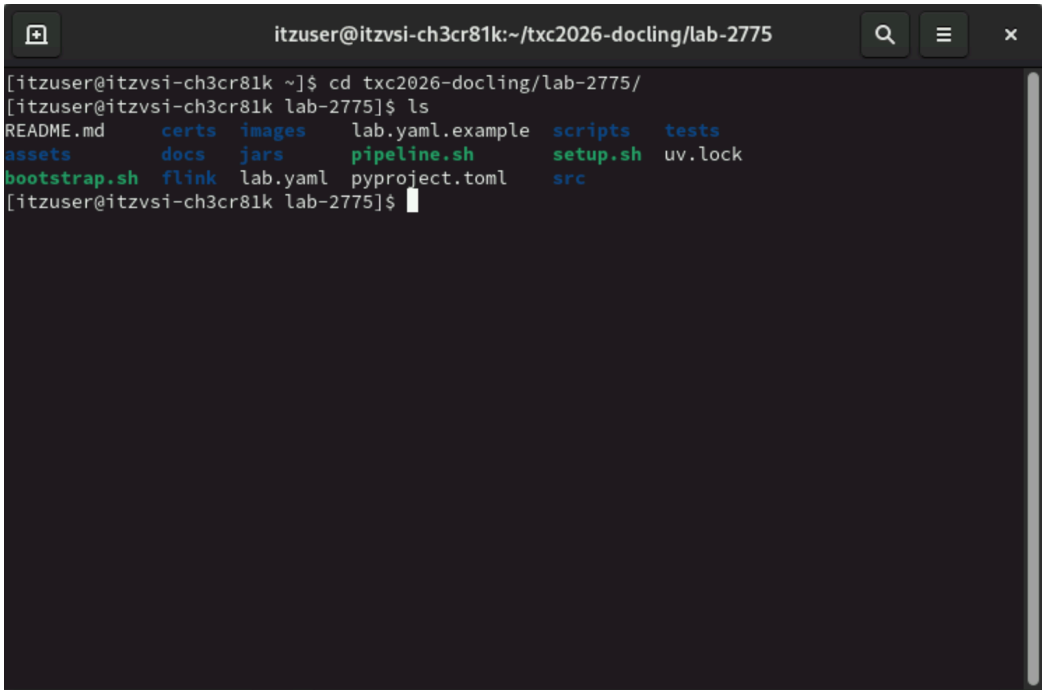


Screen 4. What the bootstrap leaves you with: two environments built, your student id and your workshop values set up, a `lab.yaml` created from the example, and the steps that follow.

Near the end it prints **you are student NN**. Write your id down in §1.5 now. If it says instead that there is **no ~/lab-2775.yaml on this VM**, raise a hand: the instructors have to put that file on your VM, and then you run the bootstrap again.

When it finishes it prints where the lab is. Open a fresh terminal — the one you ran the bootstrap in still carries the old idle timeout and will close on its own — and go there:

```
cd ~/txc2026-docling/lab-2775
```



Screen 5. The lab directory. `setup.sh` and `pipeline.sh` are the two commands you run all day; `lab.yaml` is the only file you edit.

> **NOTE**
>
> It is safe to run the bootstrap again at any point — it updates the lab and leaves your own configuration alone. If a later command is *not found*, you have a terminal that started before the tooling was installed: run `export PATH="$HOME/.local/bin:$PATH"` in it, or just open a new one.

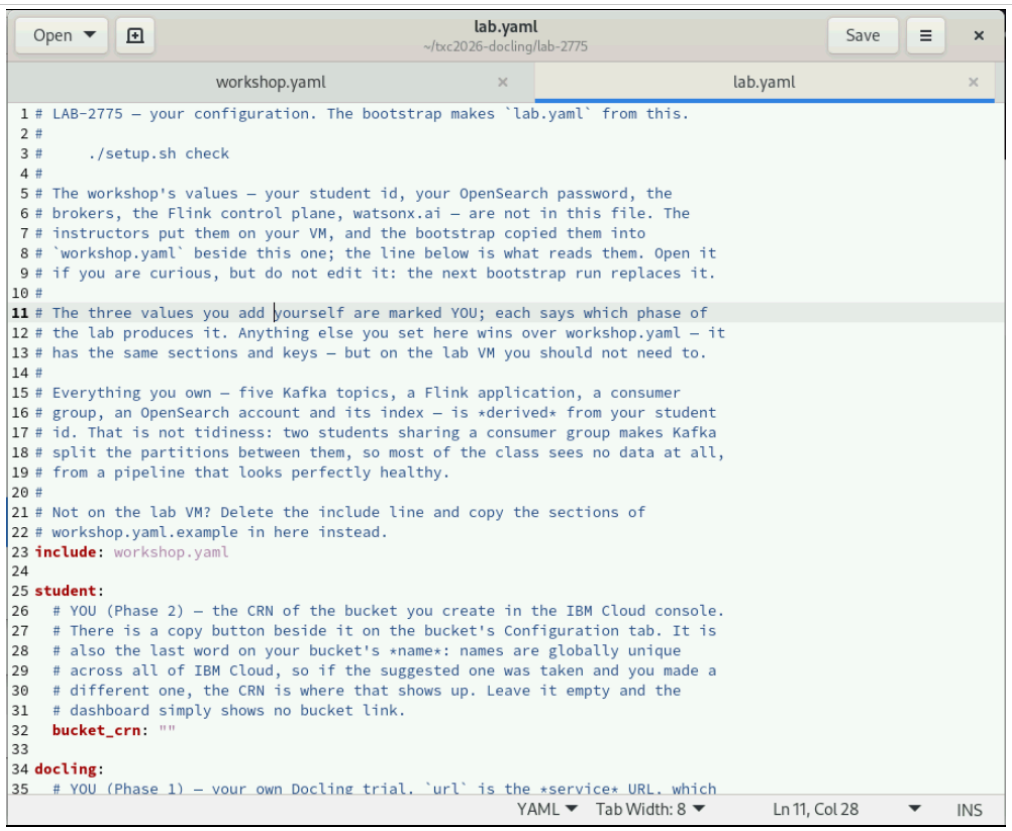
Five things there are worth knowing about now:

- `lab.yaml` — your configuration, and the only file you edit. It has just three values, and you paste each one in the moment you copy it: two in Phase 1, one in Phase 2.
- `workshop.yaml` — the workshop's values for your VM, which `lab.yaml` reads. You never edit it: a later bootstrap run replaces it.
- `./setup.sh` — creates the things your pipeline needs before it can run: your Kafka topics and your OpenSearch index. Every one of its commands ends with a **Next** block that says what to do after it.
- `./pipeline.sh` — deploys your pipeline, shows what it is doing, changes it, and opens the dashboard.
- `lab-documents/` — the four documents you will upload, listed in §1.6.

Open lab.yaml now, and keep it open

Open `lab.yaml` in an editor and leave the window open for the rest of the lab:

```
gedit lab.yaml &
```



Screen 6. `lab.yaml` in gedit. Each value you fill in is marked **YOU** and says which phase produces it. If `workshop.yaml` is open in a tab beside it, as here, it is there to read, not to edit.

Phases 1 and 2 each end with something copied out of a browser: a URL, a key, a CRN. Each goes straight into this file, between the quotes on its line, and you save with **Ctrl+S**. Nothing goes into a scratch file, so there is nothing to copy twice, and a key that the console shows only once cannot get lost.

2.4 The workshop coordinates

The Confluent cluster, the Flink control plane, OpenSearch and the container image are shared, so their addresses and credentials come from the workshop rather than from your own account. All of them are in `workshop.yaml`, which the bootstrap wrote from the instructors' file. You do not need to open it, but this is what it holds:

Value	What it is
`student.id`	your two-digit id — every name you own is derived from it
`student.opensearch_password`	your own OpenSearch account's password
`kafka.bootstrap_servers`, `api_key`, `api_secret`	the Confluent brokers, as your VM reaches them, and the SASL credentials for them
`kafka.ca_pem`	the certificate authority that signed the brokers. The bootstrap wrote it to `certs/kafka-ca.crt`
`cmf.url`, `cmf.auth`, `cmf.image`	Confluent Manager for Apache Flink: where your job is deployed, how it authenticates, and the published image it runs from
`watsonx.project_id`, `api_key`	the watsonx.ai project the embedding calls are billed to
`opensearch.hosts`	the cluster the finished chunks are indexed into

NOTE

The workshop runs more than one of these clusters and accounts, so that thirty pipelines are not all on one. Which ones you were given is decided by this file and nothing else. No step in this guide changes because of it, and your neighbour may be on a different one.

YOU NOW HAVE

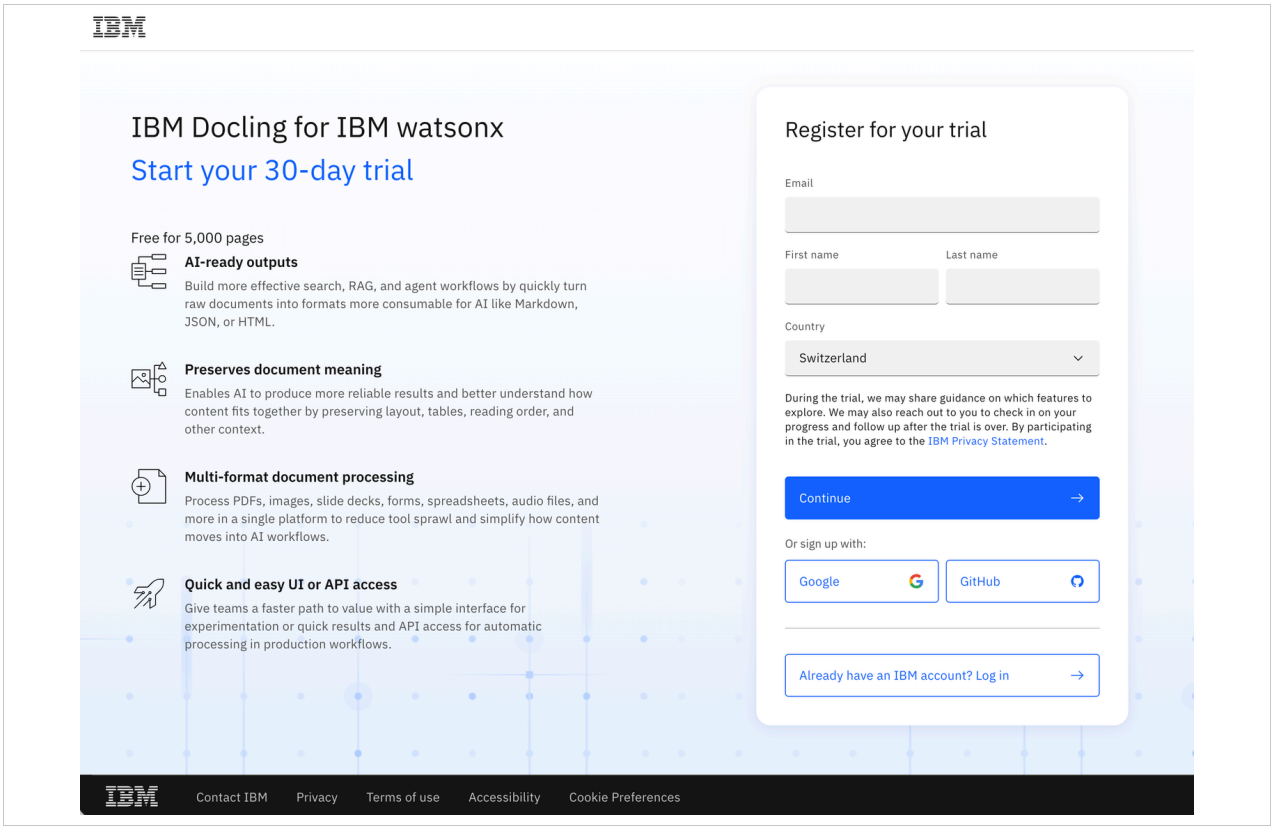
A desktop on the lab VM, the lab set up, and your two-digit id. Your IBM Cloud sign-in is waiting on the VM for Phase 2.

3 Phase 1 — Your own Docling instance

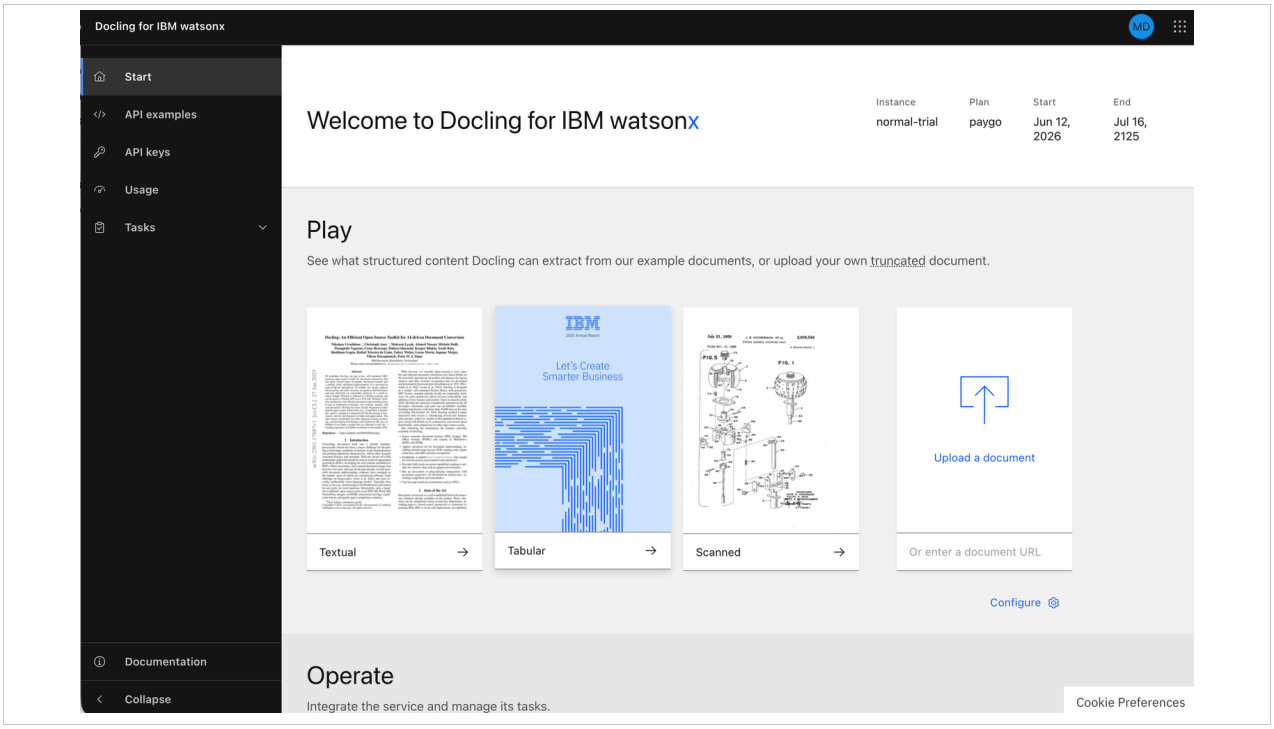
Ten minutes. You start a Docling for IBM watsonx trial, convert a document in its workbench to see what conversion actually produces, and collect the two values the rest of the lab needs: a service URL and an API key.

3.1 Start the trial

1. In the browser **on the lab VM**, go to ibm.biz/docling-for-watsonx-trial-txc26.
2. Sign in with **your own IBMid**, or create one if you do not have one. This is your personal account. It is **not** the workshop login from `ibmcloud-signin.html`, which is for Phase 2 only.
3. Follow the prompts to start the trial. Provisioning takes about a minute; wait until the instance is shown as active.
4. Open the instance. You should land in the Docling workbench.



Screen 7. The trial registration page. Registering with an existing IBMid is the **Already have an IBM account? Log in** link at the bottom of the panel.



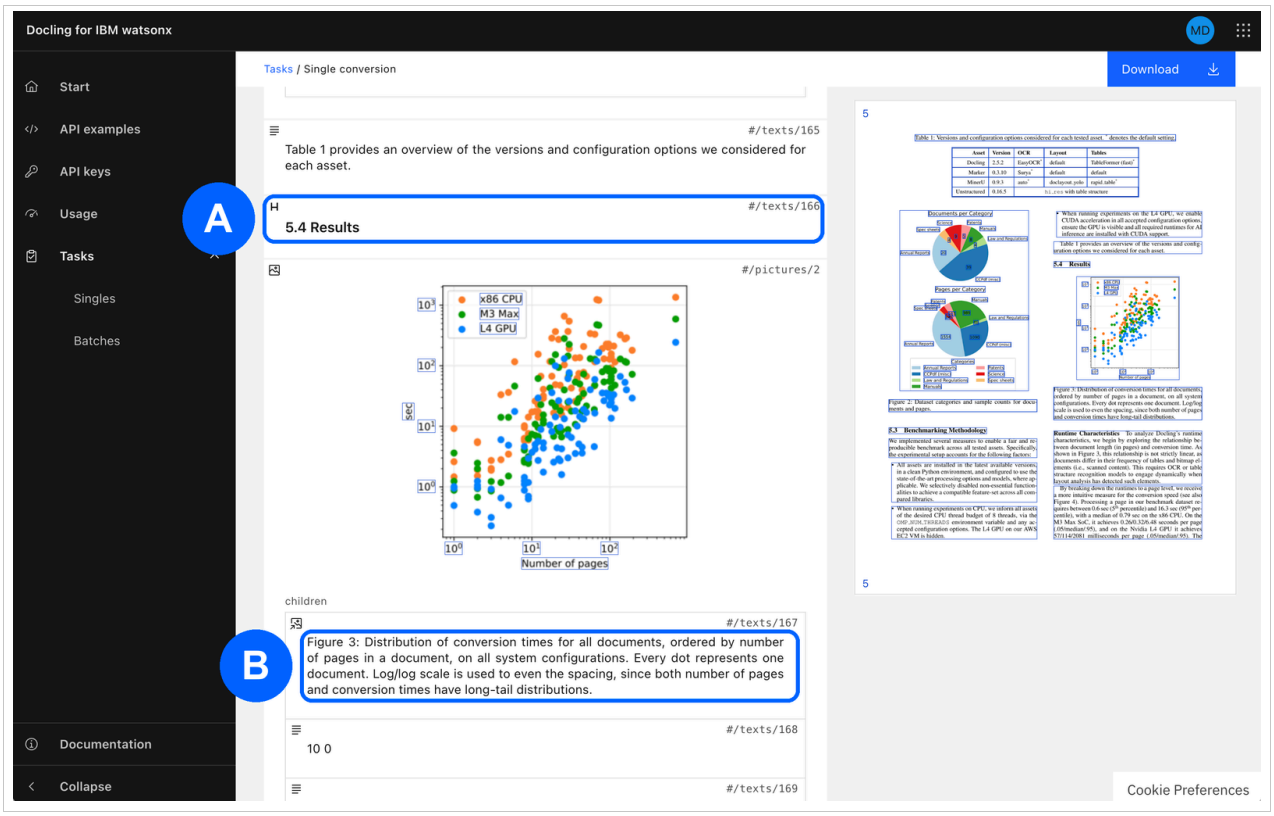
Screen 8. The workbench once the instance is active. **Play** converts a document; **API examples** and **API keys** in the sidebar are where §3.3 goes.

> **TIP**
>
> Use an ordinary Firefox window here, not a private one. Private windows are for the workshop login in Phase 2, and keeping the two logins in different windows is what stops one replacing the other.
>
> Already signed up for a trial earlier and not sure how to get back to your Docling instance? Go to ibm.biz/mydocling.

3.2 Convert a document, and look at what comes out

This step produces nothing the pipeline needs. Do it anyway: everything downstream is shaped by what conversion produces, and this is the only point in the lab where you see it directly.

1. On the workbench's **Start** page, under **Play**, either pick one of the three example documents — **Textual**, **Tabular** or **Scanned** — or upload one of your own.
2. Wait for the conversion to finish, then look at the result. Check the **reading order** on a two-column page, whether **tables** came out as tables rather than as runs of text, and whether **headings** are marked as headings.
3. Switch the output format between Markdown and JSON. The JSON is the structured document; the Markdown is a rendering of it.
4. Optional: run the **Scanned** example to see what OCR adds.



Screen 9. A conversion, both halves side by side: the recovered document on the left, item by item with its address in the structure, and the source page on the right with what was recognised drawn over it. (A) is a heading that came out as a heading; (B) is a figure's caption, recovered as a caption and kept attached to the figure above it rather than left floating in the page's text.

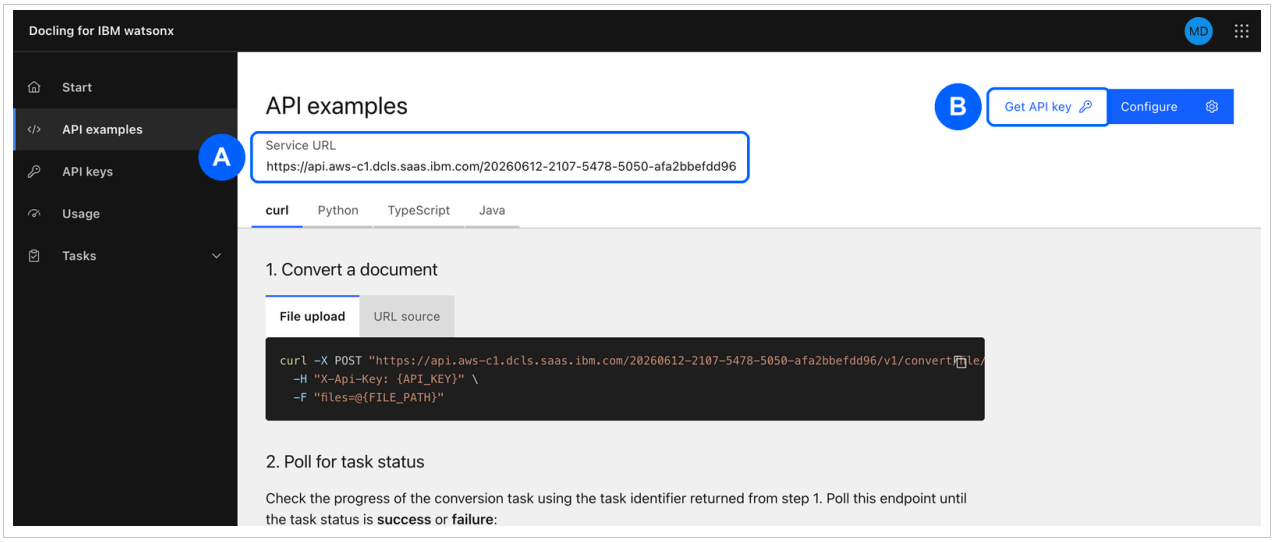
> **NOTE**
>
> The chunks your Flink job sees in Phase 3 come from exactly this structure. A heading that was recognised as a heading ends up in the chunk's `headings` field and, after enrichment, in its `heading_path` — which is what lets a retrieval system cite the section a passage came from.

3.3 Collect your service URL and API key

These two values are what your bucket's event subscription will carry in Phase 2, so that the shared trigger app submits **your** documents to **your** Docling instance.

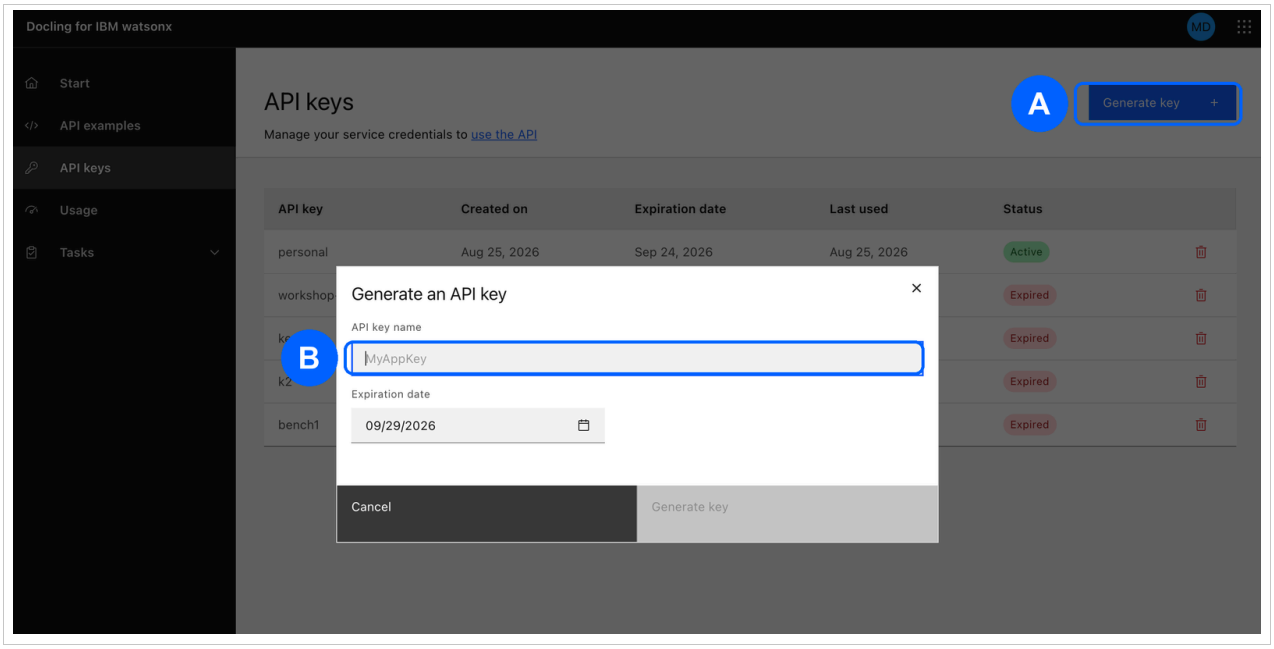
1. Open **API examples** in the sidebar. The **service URL** (A) is at the top of the page — copy the whole of it, including the path after the host: it looks like `https://api.aws-c1.dcls.saas.ibm.com/20260612-2107-…`, and the part after the host is what identifies *your* instance.
2. Switch to `lab.yaml` in gedit and paste it between the quotes on the `url:` line, under `docling:`. Save.

3. Back in the workbench, follow **Get API key** (B) — or **API keys** in the sidebar.



Screen 10. The service URL (A), and the way to the keys (B). The `curl` example below them is the same call the trigger app makes for you in Phase 2.

1. Click **Generate key**, give it a name — `txc26-lab` will do — and leave the expiration date as it is.
2. Copy the key it shows you, and paste it into `lab.yaml` between the quotes on the `api_key:` line, just below the URL. Save.



Screen 11. Generate key (A) opens the dialog; the key needs a name (B) and nothing else.

The `docling:` section of `lab.yaml` now looks like this, with your own values:

```
docling:
  url: "https://api.aws-c1.dcls.saas.ibm.com/20260612-2107-…"
  api_key: "<the key you just generated>"
```

WATCH OUT

The API key is shown **once**, which is why it goes into `lab.yaml` before the dialog is closed. If you close it without copying the key, create another one — there is no way to read an existing key back.

The address in your browser's URL bar is the **workbench**, not the service URL. They are different hostnames for the same instance, and only the service URL will work in Phase 2 — so the service URL is the only one worth copying. The dashboard works the workbench link back out of it later, which is why `lab.yaml` never asks you for it.

YOU NOW HAVE

Your Docling **service URL** and your Docling **API key**, saved in `lab.yaml`. Your bucket's event subscription in Phase 2 carries both.

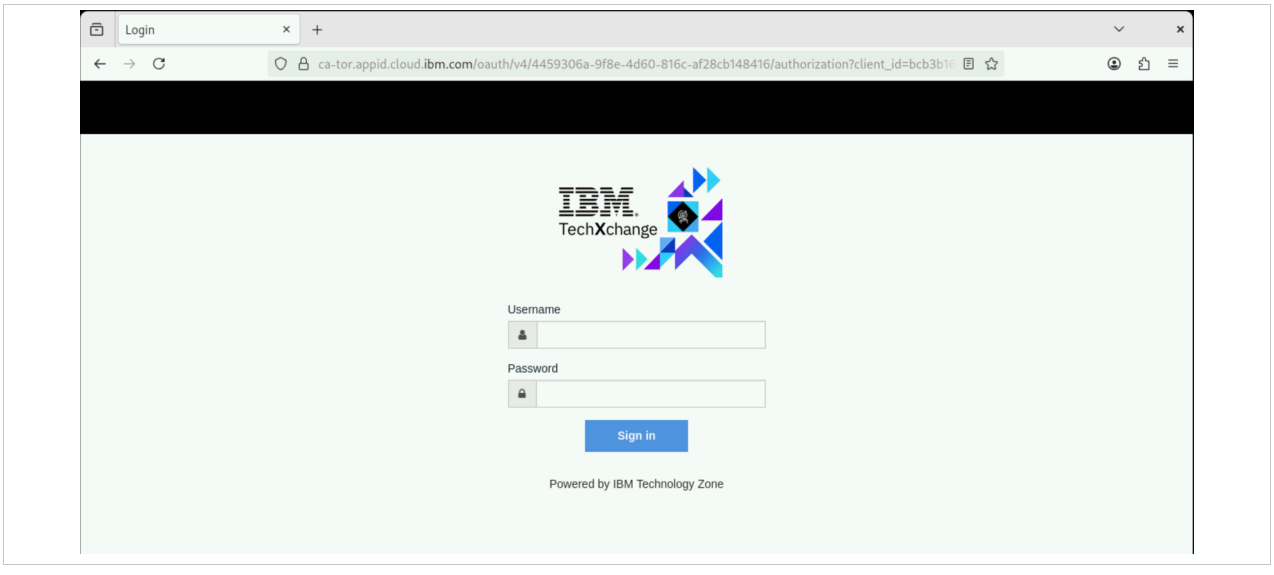
4 Phase 2 — A bucket that fires events

Fifteen minutes. You create your own bucket in the shared workshop account and subscribe it to the class's trigger app, so that writing an object to it submits that object to your Docling instance.

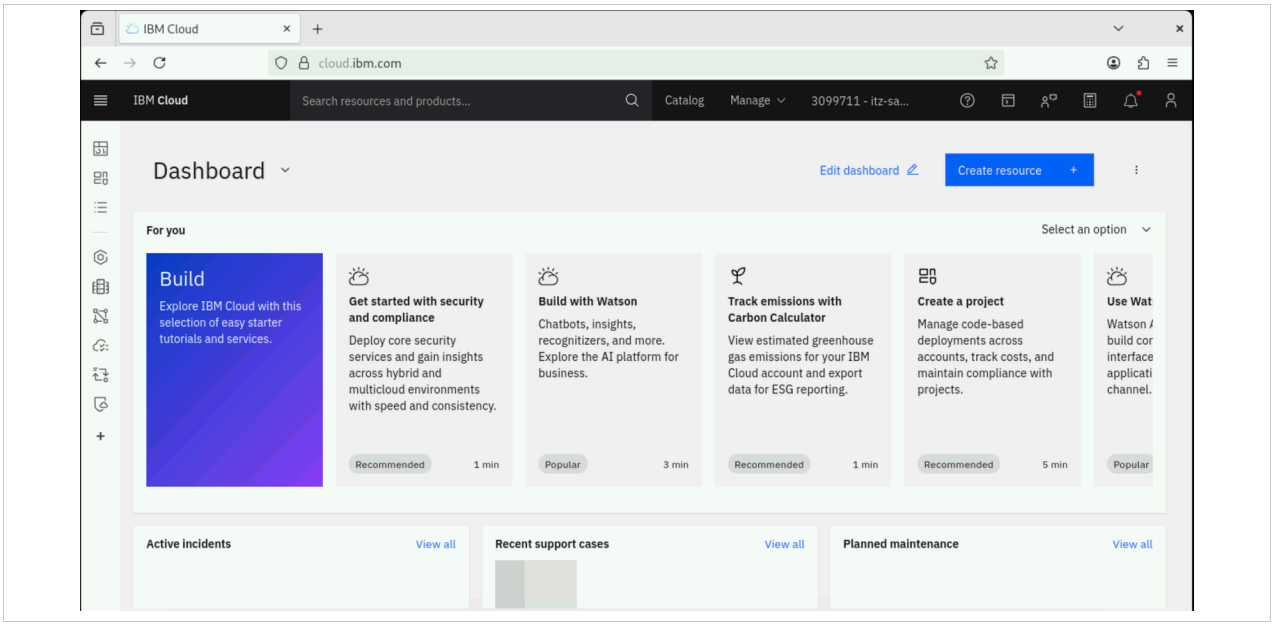
4.1 Sign in to IBM Cloud

This is the second of the two logins from §2.2: the workshop's, not your IBMid. Your sign-in is already on the VM, as a page in your home folder.

1. Open **Files** from the dock. In your **Home** folder, double-click `ibmcloud-signin.html`. It opens in Firefox, showing your sign-in link, your username and your password, with a **Copy** button beside each.
2. **Right-click** the **Workshop sign-in** button and choose **Open Link in New Private Window**. It is that link, and not `cloud.ibm.com`, that puts you in the workshop account. A plain click only reminds you to do this.
3. In the private window, sign in with the username and password from the page. Copy them across rather than typing them. **Show** reveals the password if you want to check it. Do **not** use your IBMid here.
4. Stay in the private window for the rest of Phase 2. If the console greets you with your own name, you are in your IBMid's window: close it and start again from step 2.



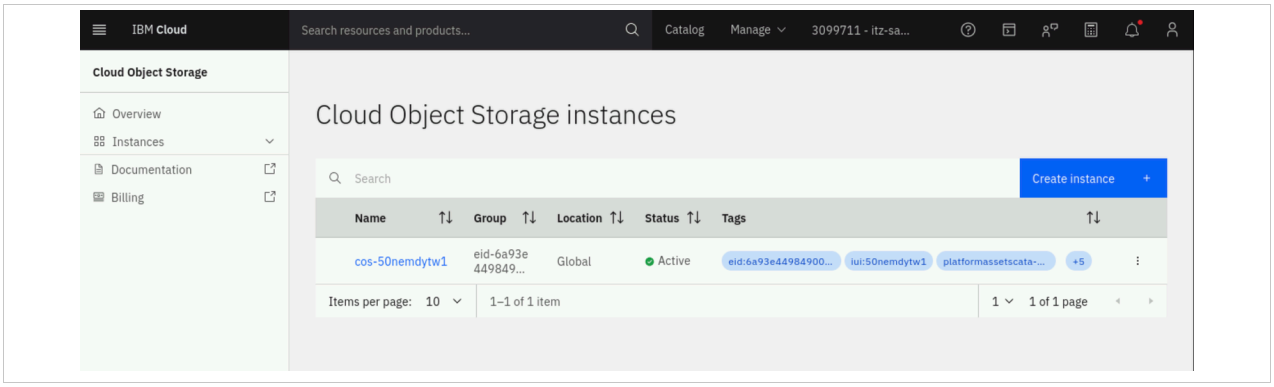
Screen 12. The sign-in the authorization link takes you to. It is a workshop login, not the ordinary IBM Cloud one — no IBMid is involved.



Screen 13. Signed in with the workshop login, in the private window. Everything in Phase 2 happens here.

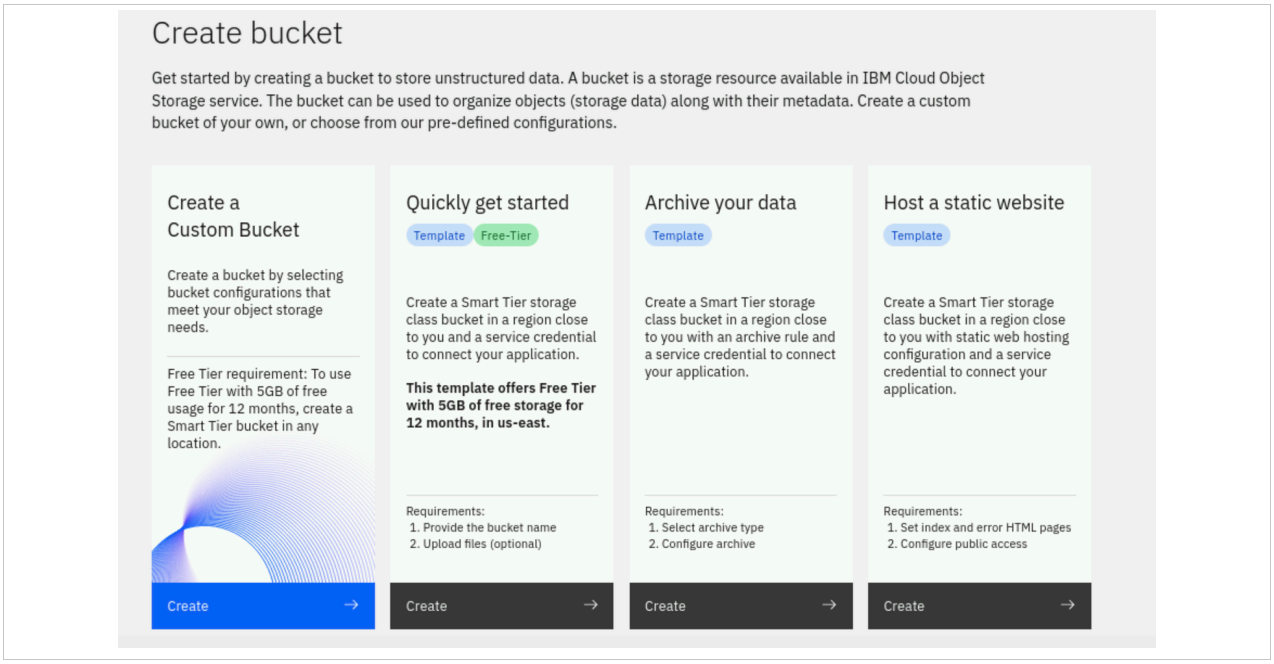
4.2 Create your bucket

Go straight to cloud.ibm.com/objectstorage/instances. There is exactly one instance in this account — open it.

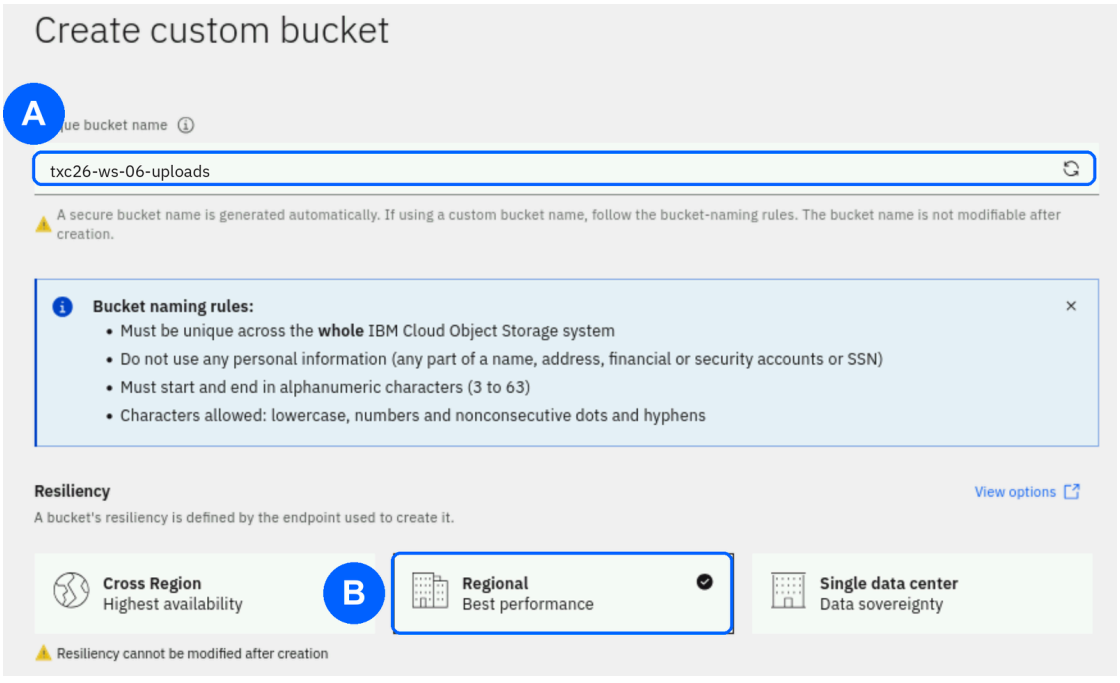


Screen 14. One instance, shared by the whole class. Your bucket goes inside it.

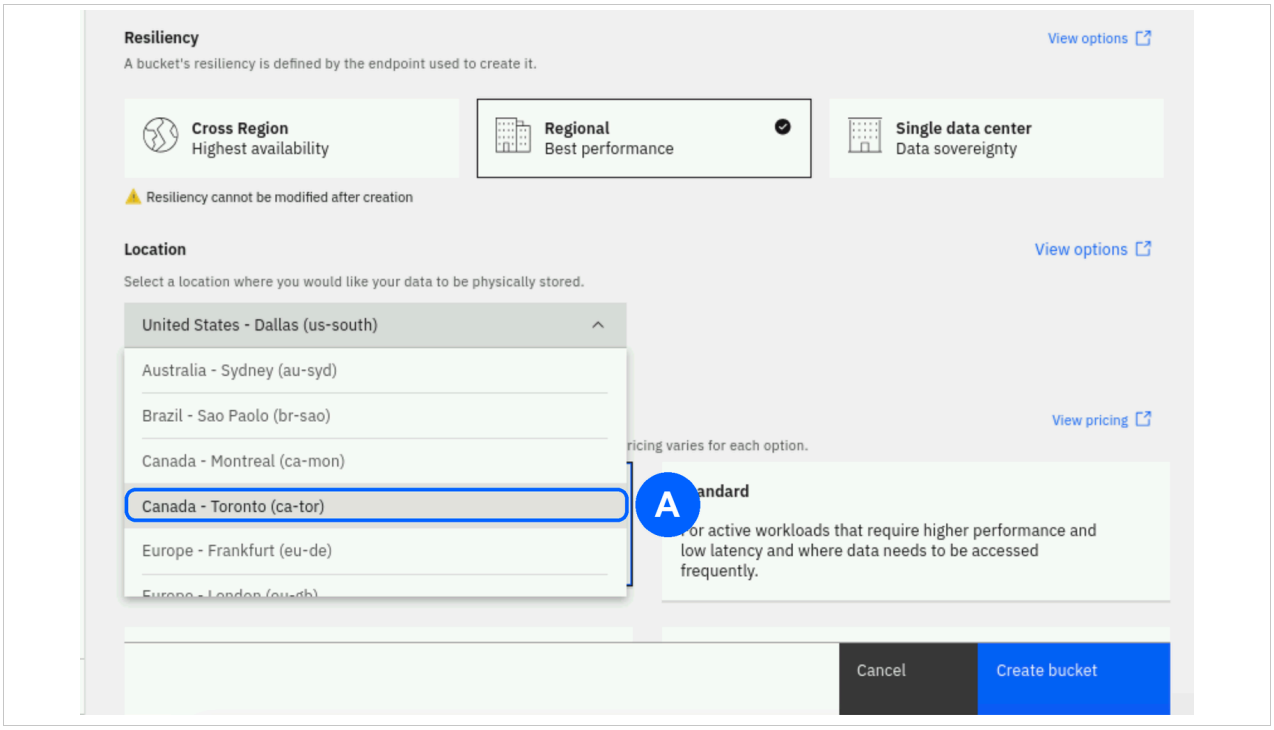
1. On the **Buckets** tab, click **Create bucket**.
2. Choose **Create a Custom Bucket** — not one of the templates beside it, which pick their own region.
3. Name it `txc26-ws-NN-uploads`, substituting your two-digit id.
4. Set **Resiliency** to **Regional** and **Location** to **Canada – Toronto (ca-tor)**, which is where the rest of the workshop runs. Leave the storage class and every other setting at its default.
5. Click **Create bucket**.



Screen 15. Four ways to make a bucket. Take the custom one on the left — each of the three templates picks a region of its own, and this lab needs `ca-tor`.



Screen 16. The bucket's name (A) and its resiliency (B). Both are fixed at creation — a wrong name means creating a second bucket, not renaming this one.

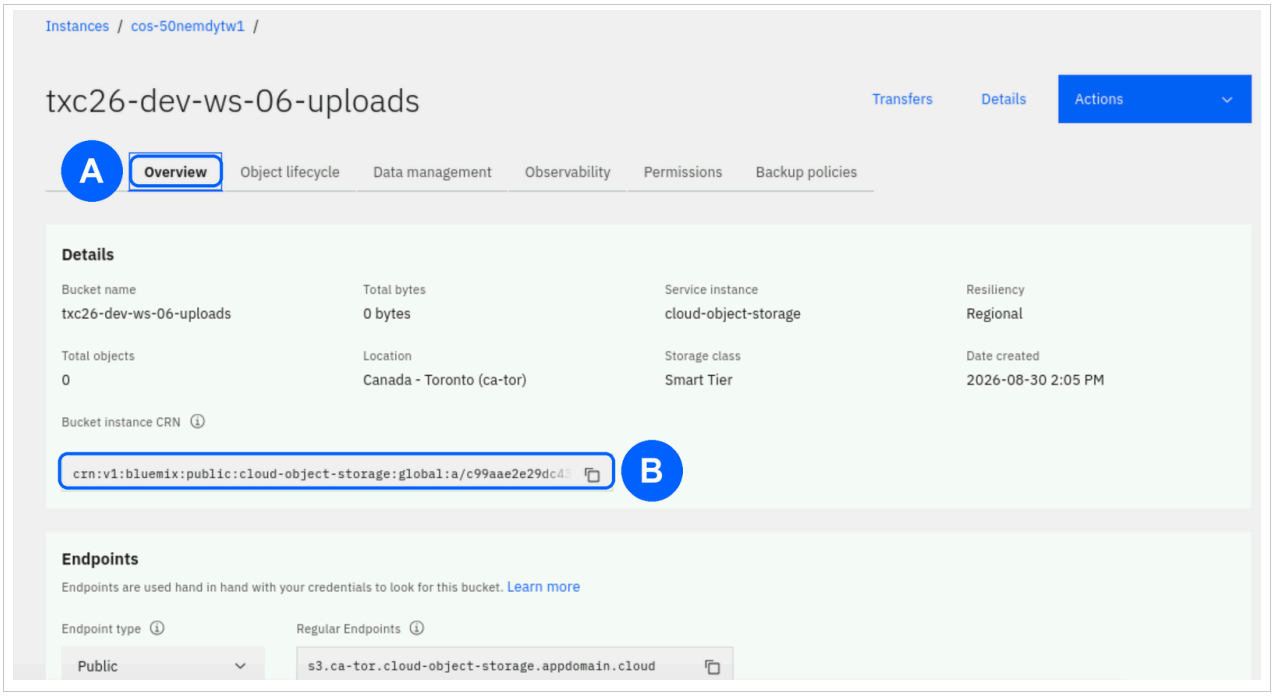


Screen 17. Canada – Toronto (ca-tor) (A), further down the same form.

WATCH OUT

Bucket names are globally unique across all of IBM Cloud, not just within your account. If `txc26-ws-NN-uploads` is taken, append your initials — `txc26-ws-06-uploads-md` — and use that name everywhere below.

1. Open your new bucket and go to its **Overview** tab.
2. Copy the **bucket instance CRN** — there is a copy button beside it — and paste it into `lab.yaml` between the quotes on the `bucket_crn:` line, under `student:`. Save. That is the last of the three values: the dashboard links to your bucket from it, and `./setup.sh info` reads your bucket's name out of it.

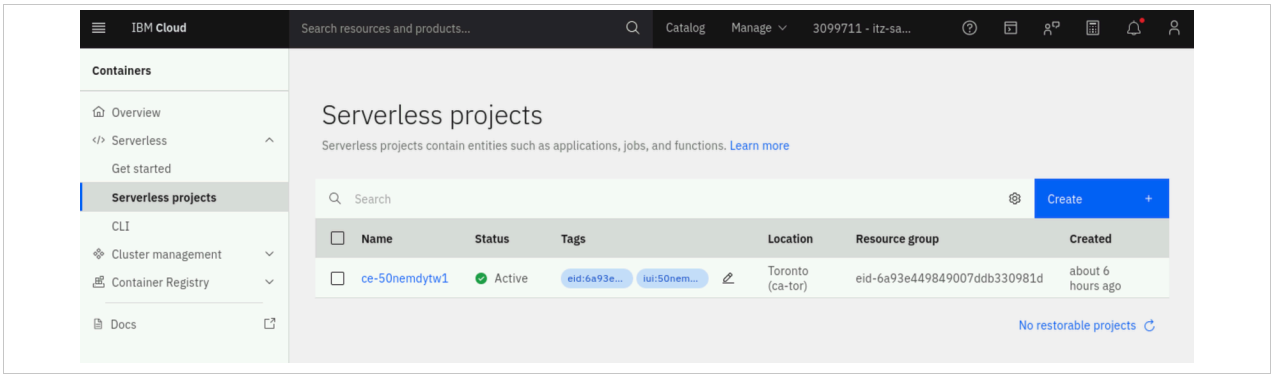


Screen 18. The **Overview** tab (A) and the bucket instance CRN (B). This tab is also where to confirm the location came out as `ca-tor`.

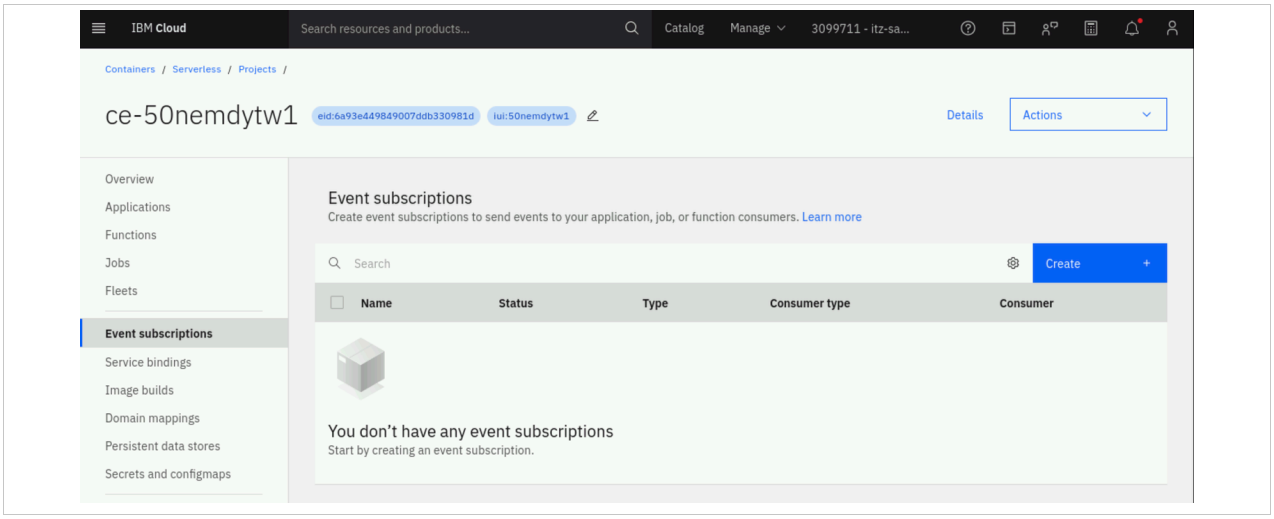
4.3 Subscribe your bucket to the trigger app

The app that reacts to the event, `docling-trigger`, is deployed once for the whole class and stores nothing about anybody. The three things that differ between students — your Docling URL, your Docling key and your chunk topic — travel **with each event**, as attributes on your own subscription. That is what lets one small deployment serve thirty different Docling instances.

Go to cloud.ibm.com/containers/serverless/projects and open the one project listed. Then choose **Event subscriptions** in the left-hand navigation and click **Create**.



Screen 19. One Code Engine project, in `ca-tor`, holding the trigger app the whole class shares.



Screen 20. Event subscriptions belongs to the *project*, not to the application — the application is chosen later, on the wizard's third step.

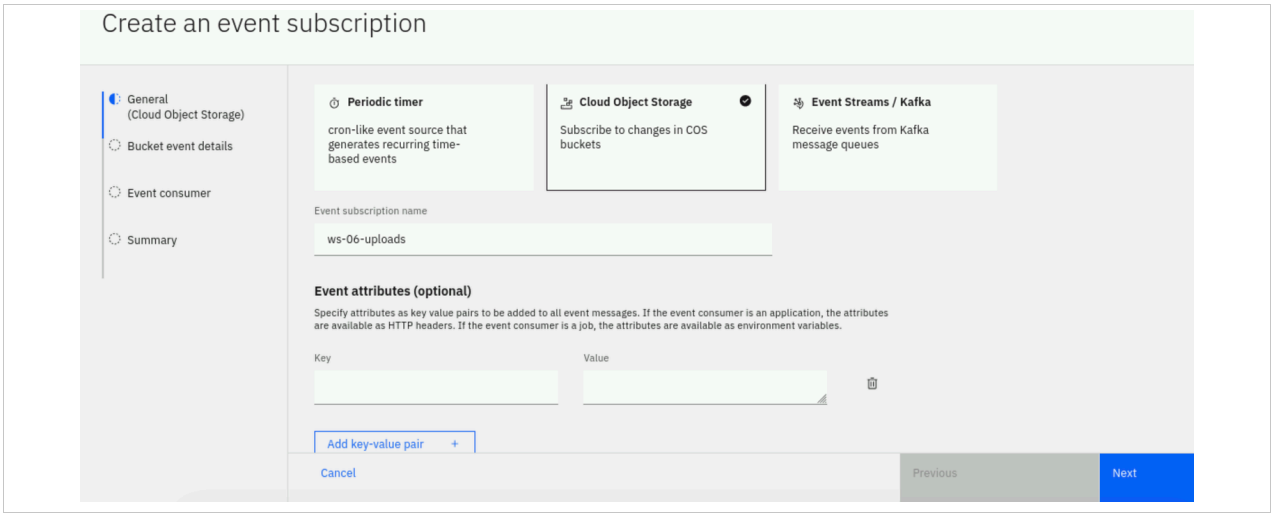
The values the wizard asks for are already on your VM. In the terminal, run:

```
./setup.sh info
```

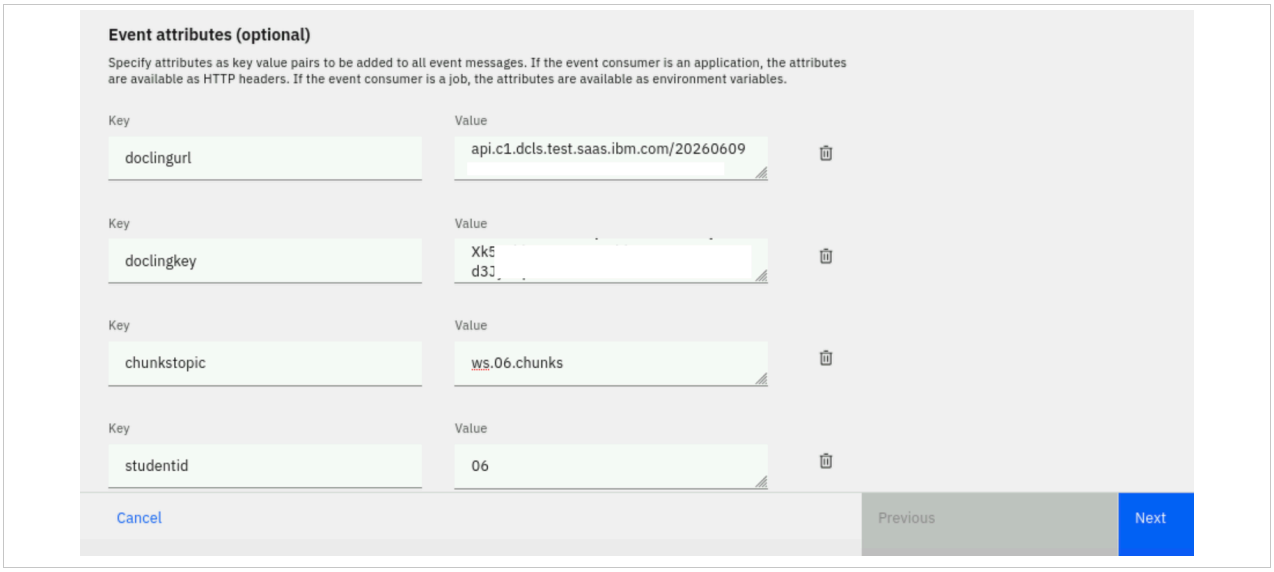
Its last block is this form, filled in from your `lab.yaml` and your id, ready to copy from. Only the Docling key is left out, because the output gets shared on screen: copy that one from `lab.yaml`.

The wizard has four steps. On **General**, choose **Cloud Object Storage** as the source, name the subscription `ws-NN-uploads`, and add four **event attributes**. Those four are what tell the shared app to send *your* documents to *your* Docling instance:

Key	Value
`doclingurl`	your Docling **service** URL, from §3.3 — the whole thing, path included. `docling.url` in your `lab.yaml`
`doclingkey`	your Docling API key, from §3.3. `docling.api_key` in your `lab.yaml`
`chunkstopic`	`ws.NN.chunks`, with your two-digit id
`studentid`	your two-digit id



Screen 21. Step one: the source, the name, and **Add key-value pair** for each attribute.

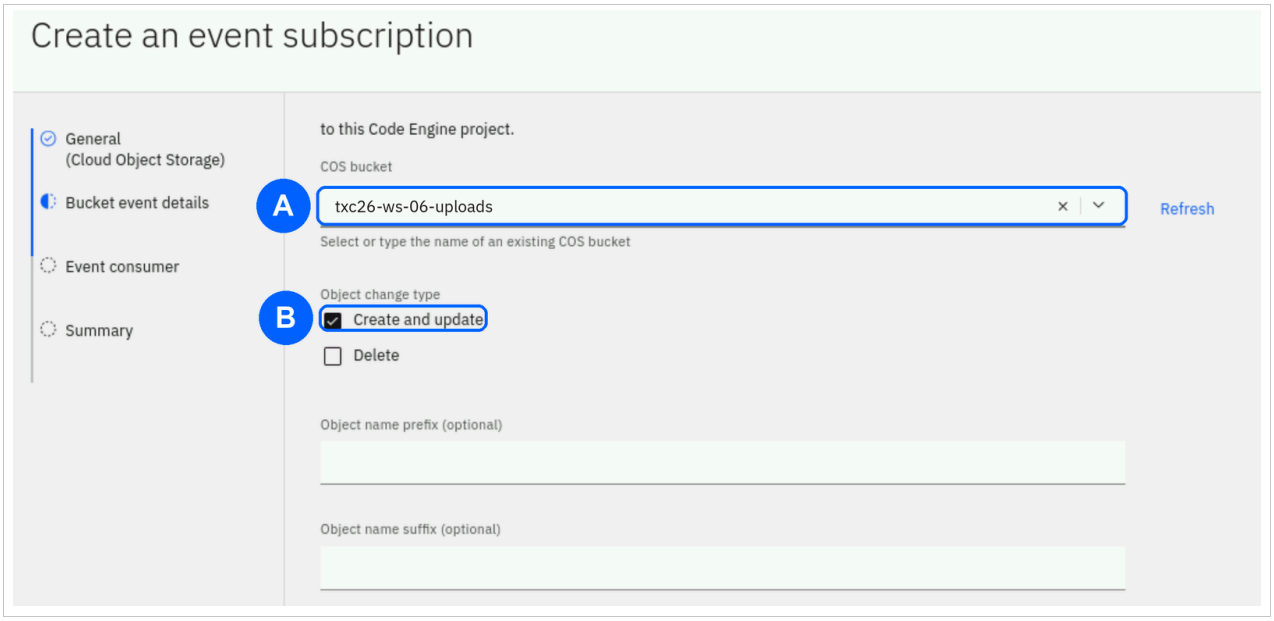


Screen 22. All four attributes filled in. They travel with every event this bucket raises.

> **WATCH OUT**
>
> Type the attribute names exactly as they are written: `doclingurl`, `doclingkey`, `chunkstopic`, `studentid`. They become CloudEvent extension attributes and the specification allows only lowercase letters and digits, so `docling-url` is not a legal name and will be rejected.

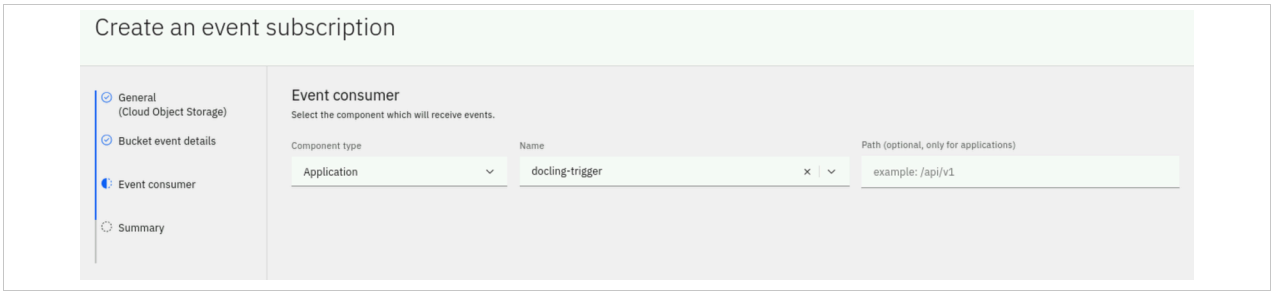
On **Bucket event details**, pick your bucket from §4.2 and leave **Create and update** ticked — that is the write event. Leave **Delete** unticked, and both name filters empty.

> **NOTE**
>
> The bucket list may stay empty and show **Some buckets could not be loaded**. That is the console, not your permissions, and it is harmless: type your bucket's name into the field. The name is the last part of the CRN you saved in `lab.yaml`, after `:bucket:`.



Screen 23. Your bucket (A), and the only change type this lab wants (B).

On **Event consumer**, choose component type **Application** and the application called `docling-trigger`. Leave the path empty. Then **Summary**, and create it.



Screen 24. One application serves the whole class; what differs between students is the four attributes, not this.

Check that the subscription's status becomes **Ready**. If it does not, the message beside it usually names the problem — most often a bucket that is not in the instance the project can reach.

> **NOTE**
>
> Your Docling API key is stored in **your** subscription, not in the shared app — nobody else's events can reach your Docling instance, and the app itself never holds a credential of yours.

4.4 Do not upload yet

The subscription is live, so an upload right now would start a real conversion — and Docling would then try to produce to a topic that does not exist yet. The brokers in this lab do not create topics on demand, so those chunks would be lost.

Create the topics first. That is the first step of Phase 3, and it takes one command.

YOU NOW HAVE

Your own bucket, and a subscription that turns every write to it into a submission to your Docling instance. From here on, uploading a file is the only thing you do by hand.

5 Phase 3 — See the chunks arrive

Fifteen minutes. You check your configuration, create your five topics and your index, start the inspection dashboard, and upload your first document — then watch the chunks land on Kafka with nothing consuming them yet.

5.1 Check your configuration

Your configuration is one file, `lab.yaml`, and you have filled it in as you went. The workshop's values are not in it: its first line, `include: workshop.yaml`, reads them from the file the bootstrap set up. The three marked **YOU** should all be there by now. Compare it with this in gedit, and make sure it is saved:

```
include: workshop.yaml

student:
  bucket_crn: "crn:v1:bluemix:public:cloud-object-storage:global:a/…:bucket:txc26-
ws-NN-uploads"

docling:
  url: "https://api.aws-c1.dcls.saas.ibm.com/20260612-2107-…"
  api_key: "<your Docling API key>"
```

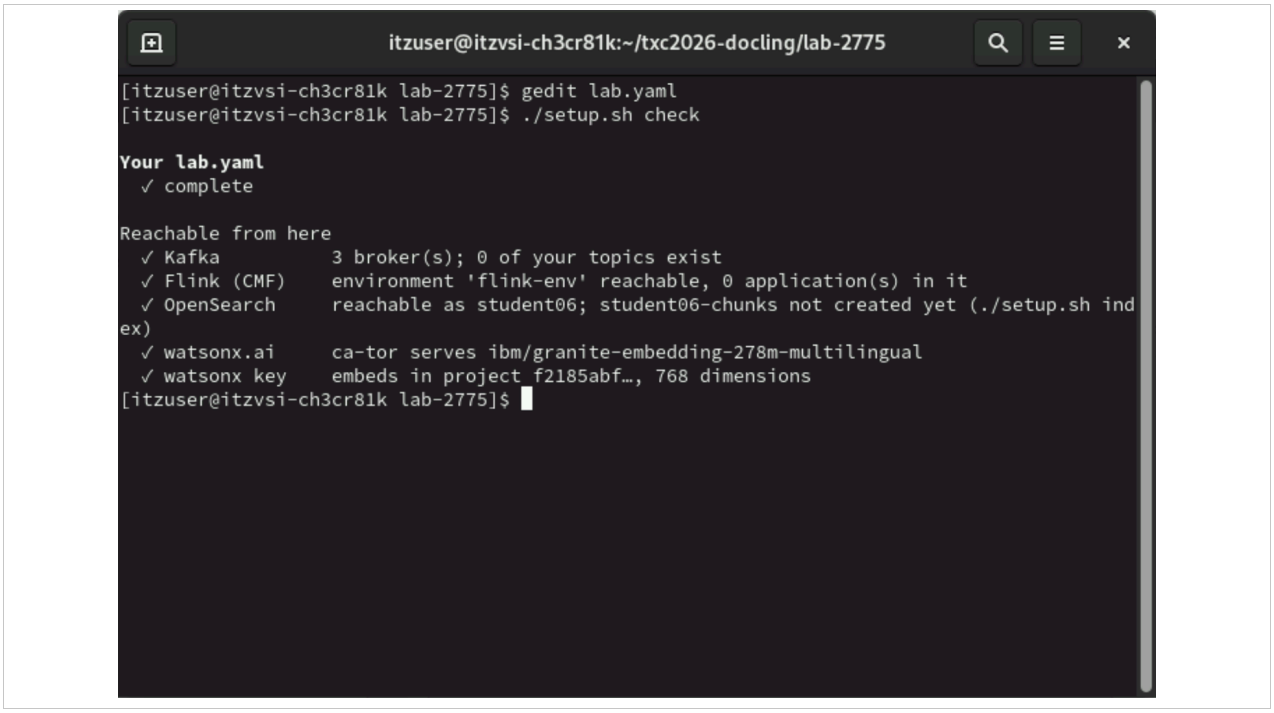
NOTE

Your student id is what makes everything yours, and it comes from `workshop.yaml`. Every topic name, application name, consumer group and index name is derived from it, so you never type any of them. Do not add an `id:` line to `lab.yaml`: a value you set there wins over the workshop's, and that is how two students end up sharing a topic.

Then check it, which is worth thirty seconds now and can save twenty minutes later:

```
./setup.sh check
```

It first tells you whether anything is missing, naming the line to fix. A missing value that comes from `workshop.yaml` says so, and that one is for the instructors to fix, not you. Then it tries to reach each of the systems the lab uses and prints one line each. Last comes its **Next** block, which here should tell you to create your topics. If it sends you back to Phase 1 or Phase 2 instead, one of your three values is missing, or `lab.yaml` is not saved.



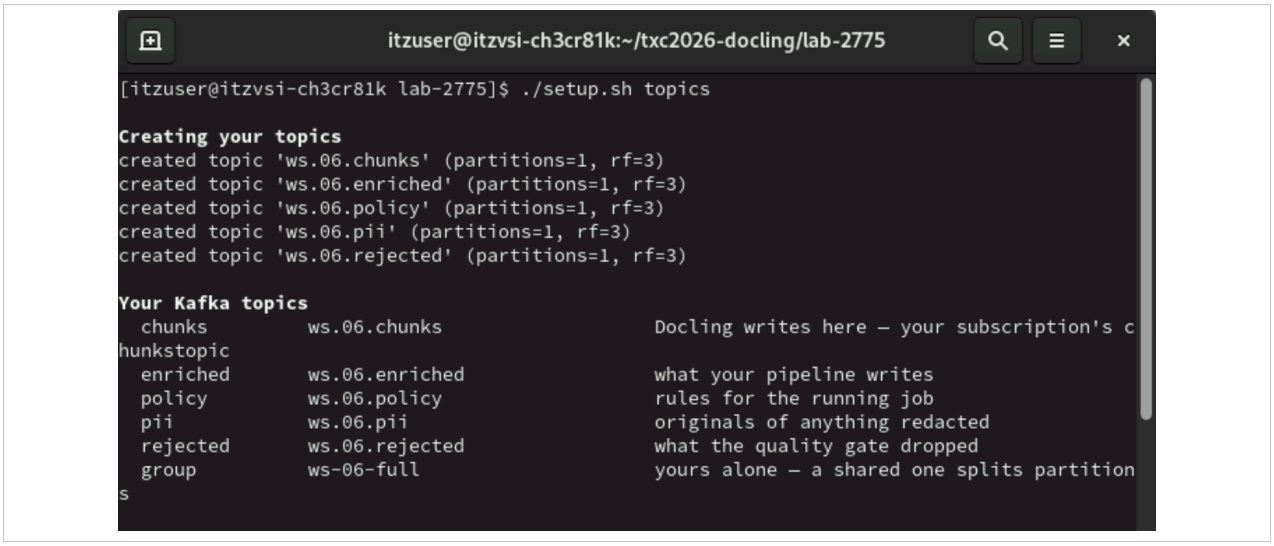
Screen 25. A `lab.yaml` that is complete, and five systems that answer. Everything that follows depends on these lines.

> **NOTE**
>
> The lines are independent — a ✗ on OpenSearch does not stop you deploying a pipeline. If one of them fails, fix it before moving on: each of these failures otherwise shows up much later as a pipeline that looks perfectly healthy and produces nothing.

5.2 Create your topics

```
./setup.sh topics
```

That creates five topics named after your id, then prints them back together with your bucket's name, your index, and the subscription values from Phase 2. Everything the rest of the lab has to be given is in that block — `./setup.sh info` prints the same block on its own, without creating anything.



Screen 26. Five topics created, then read back with what each one carries. `group` is your consumer group, derived from your id like everything else.

The five topics:

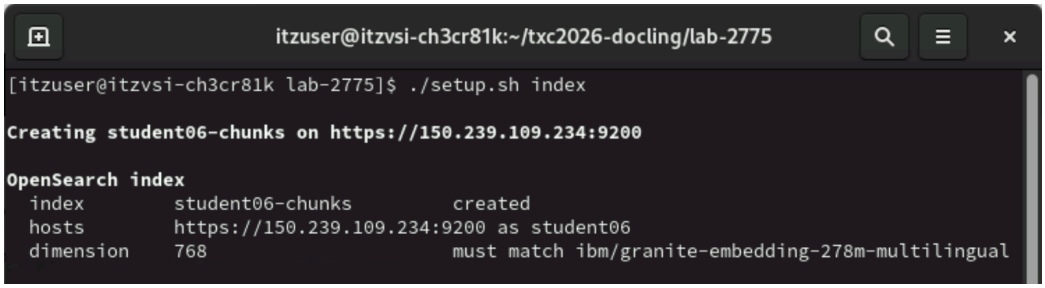
Topic	Carries	Used by
`ws.NN.chunks`	the chunks Docling produces	everything, from Phase 3 on
`ws.NN.enriched`	the finished records	everything, from Phase 4 on
`ws.NN.policy`	rules for the running job	Phase 5
`ws.NN.pii`	the original text of chunks that contained PII	Phase 5
`ws.NN.rejected`	chunks the quality gate dropped	Phase 5

> **NOTE**
> Why per-student topics, and per-student consumer groups? Because if two students shared a consumer group on one topic, Kafka would **split the partitions between them** rather than give each a copy — and most of the class would see a healthy green job producing nothing at all. The scripts derive both names from `WS_ID` so that cannot happen.

5.3 Create your OpenSearch index

```
./setup.sh index
```

Both pipelines in this lab write their finished records to **two** places: your `ws.NN.enriched` topic and an OpenSearch index called `studentNN-chunks`. The index has to exist first, and it has to be created with the right vector dimension — 768, matching the embedding model — because a k-nearest-neighbour field's dimension is fixed when the index is made.



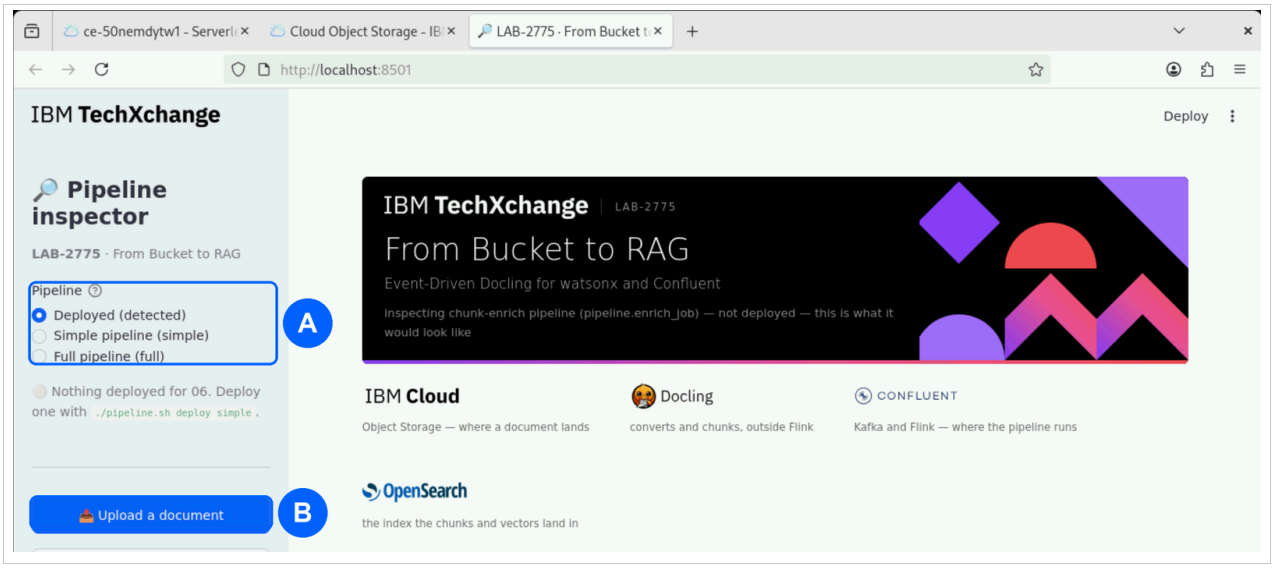
Screen 27. The index, its host and the dimension it was created with. That number has to match the model the embed stage calls.

> **NOTE**
>
> This is a separate command from `./setup.sh topics` because it talks to a different system with a different credential. If it stops because `student.opensearch_password` is empty, your workshop values are missing: ask the instructor.

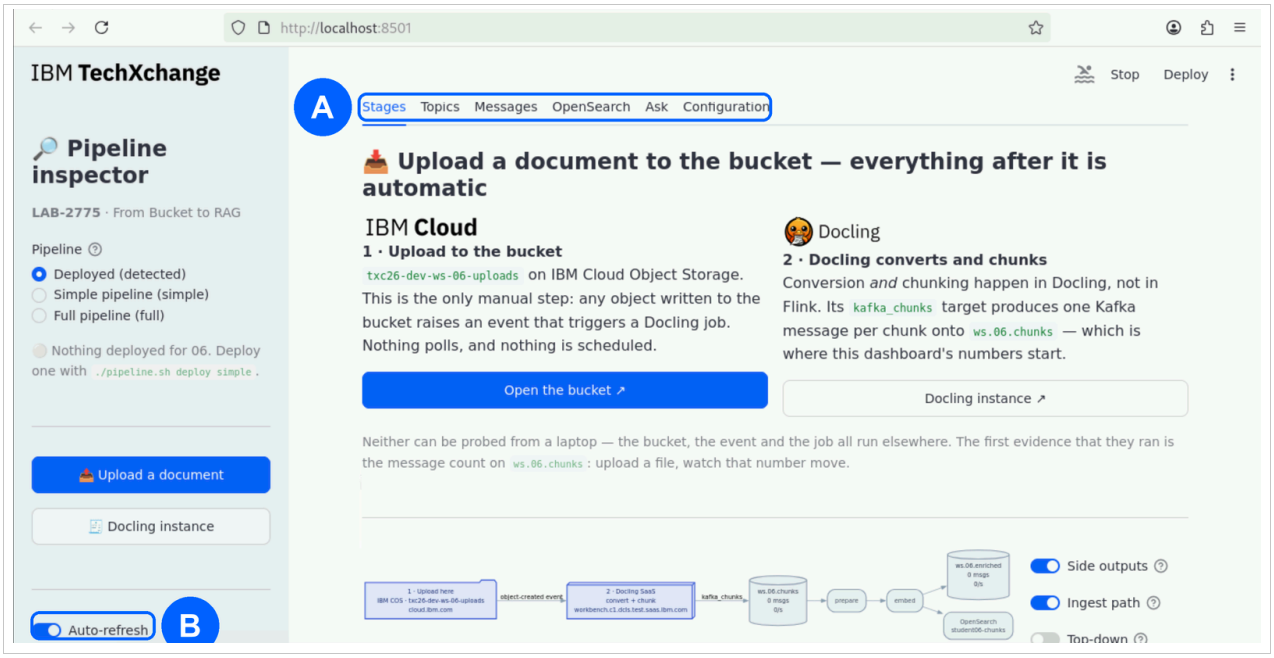
5.4 Start the inspection dashboard

```
./pipeline.sh inspect
```

It opens localhost:8501 in the VM's browser, in an ordinary window: the dashboard runs on your VM and needs no login. **Leave it running** and open a second terminal for everything that follows.



Screen 28. The dashboard on first load. The pipeline selector (A) says nothing is deployed, which is correct — you deploy one in Phase 4. **Upload a document** (B) shows the way to your own bucket.



Screen 29. Further down the same page: the tab bar (A) that the rest of the lab lives in, and the auto-refresh toggle (B) at the foot of the sidebar.

What you are looking at:

Control	What it tells you
Pipeline (sidebar)	which of the two pipelines is deployed. It asks the control plane rather than guessing. Right now it will say nothing is deployed — that is correct.

Control	What it tells you
Upload a document (sidebar)	the link to your bucket, and how to open it in the private window where the workshop login is, with the list of which document to upload in which phase. Uploading is the only thing you do that changes anything.
Stages tab	the pipeline drawn stage by stage, coloured by what each one is doing, with the two ingest steps in front of it.
Topics tab	how many messages are on each topic and how fast they are arriving.
Messages tab	the actual records on any topic, as a table and as raw JSON.
OpenSearch tab	how many documents are in your index, how big it is, and a search box over them. From Phase 4 on, this is where the pipeline's output ends up.
Ask tab	a question answered from your index — retrieval and generation, over the chunks your own pipeline put there. Phase 4 ends here.
Configuration tab	what the dashboard resolved from your `lab.yaml`, with secrets masked. The first place to look when something is pointed at the wrong thing.

NOTE

The Topics tab reports consumer lag as *not reported*, deliberately. A Flink source only commits offsets back to its consumer group on a completed checkpoint, and these lab jobs run without checkpointing — so the group stays empty and any lag number would be pure fiction. Watch the **rates** instead.

5.5 Upload a document and watch the chunks arrive

1. Turn on **Auto-refresh** in the sidebar and leave it at 5 seconds. It refreshes the Stages, Topics and OpenSearch tabs, and pauses by itself on Messages, Ask and Configuration, so a refresh never clears what you are reading there. **Refresh now** works on any tab.
2. Click **Upload a document**. It does not open the console in this window, because your bucket is in the workshop account and that login is in your **private** window. **Right-click** the bucket link in the panel and choose **Open Link in New Private Window**, or copy the link from the panel into the private window you already have.

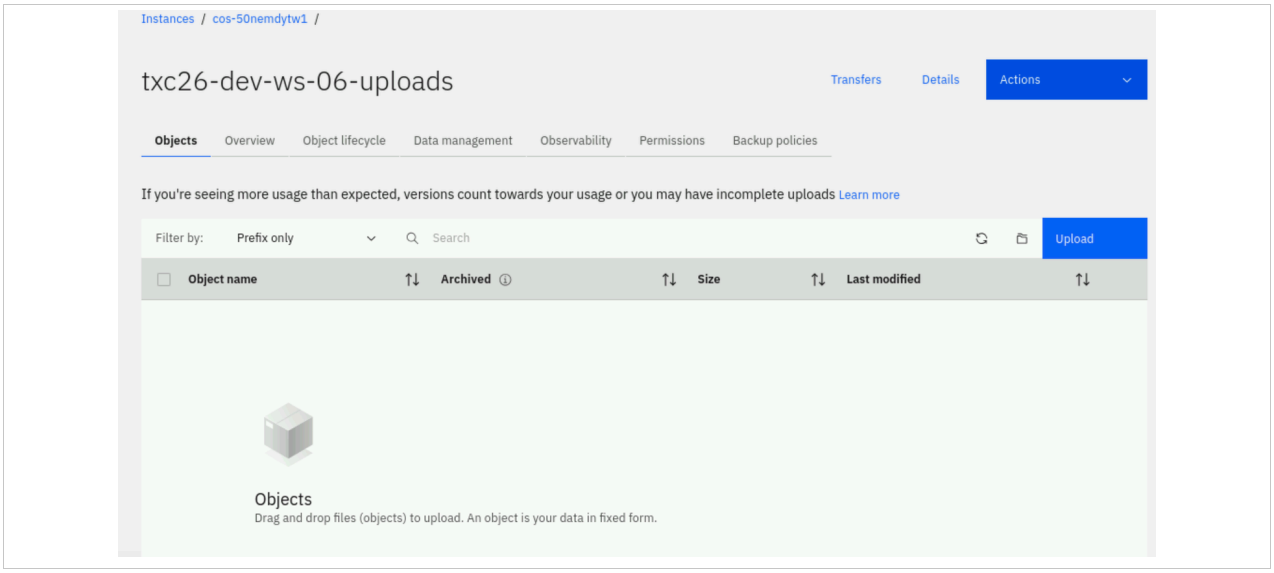


UPLOAD NOW

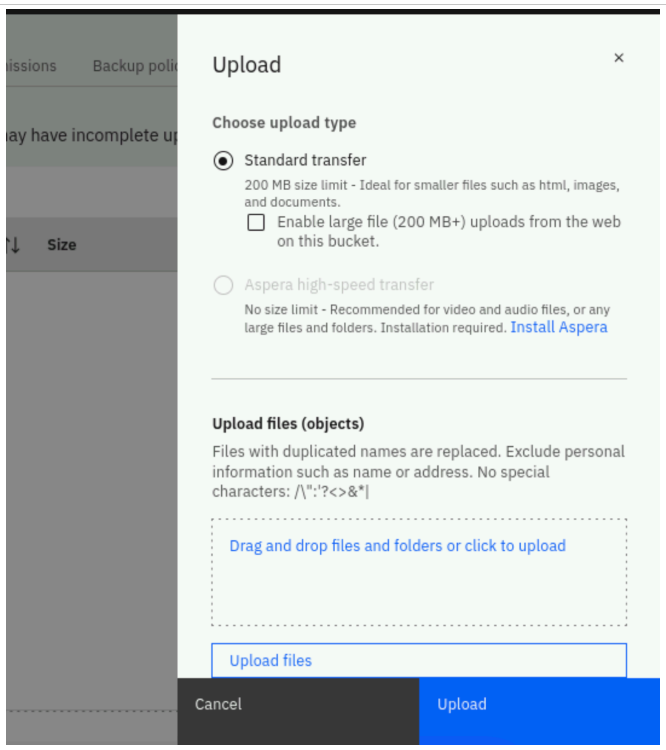
`lab-documents/01-technical-note.pdf`

Phase 3, your first document. Drop it on your bucket's **Upload** panel, in the private window.

3. Go back to the dashboard, to the **Topics** tab, and watch `ws.NN.chunks`.



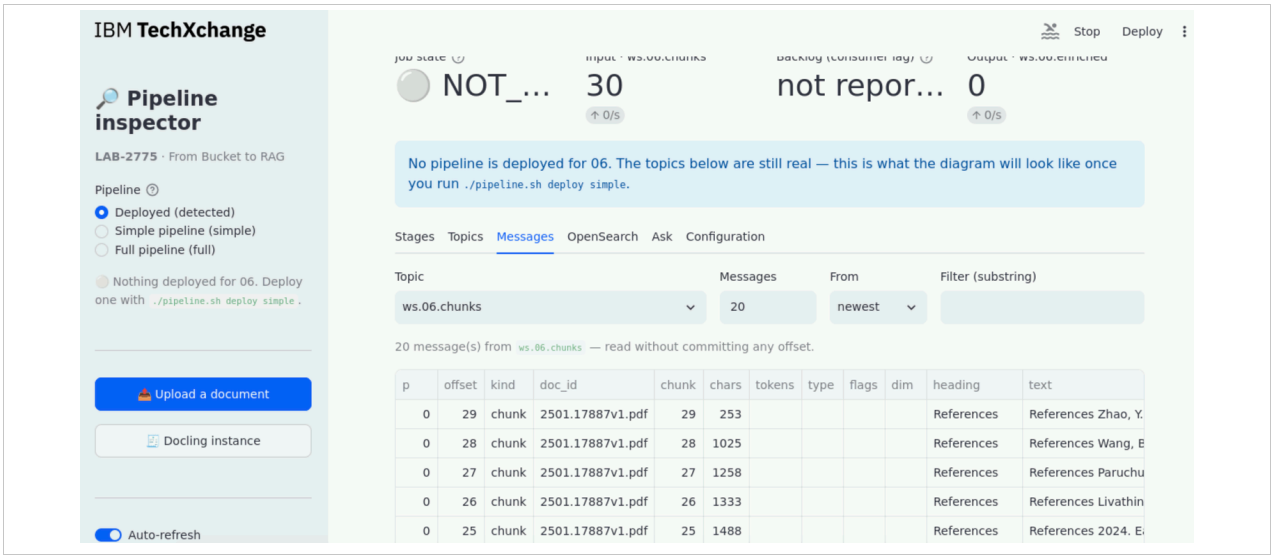
Screen 30. Your bucket, empty, on its **Objects** tab. **Upload** is at the right of the filter bar.



Screen 31. The bucket's upload panel. **Standard transfer** and drag-and-drop; nothing else on it matters for a document of this size.

Within twenty to sixty seconds the count goes from zero to however many chunks your document produced. Nothing you did made that happen: the write raised an event, the event reached the trigger app, the app submitted the object to your Docling instance, and Docling produced the chunks itself.

4. Open the **Messages** tab, choose `ws.NN.chunks`, and read from *oldest*.
5. Expand one record and read it.



Screen 32. The Messages tab reading `ws.NN.chunks`. One row per chunk, with the columns a later stage will fill in — `tokens`, `type`, `dim` — still empty.

A chunk record looks like this — and this exact shape, with fields added to it, is what flows through every stage of the rest of the lab:

```
{
  "text": "1 Summary\nSince the autumn of 2025, field engineers ...",
  "doc_id": "01-technical-note",
  "chunk_index": 1,
  "chunk_id": "49eb...",
  "headings": ["1 Summary"],
  "page_numbers": [1],
  "metadata": {"origin": {"filename": "01-technical-note.pdf",
                          "mimetype": "application/pdf",
                          "binary_hash": 13823681836276983029}}
}
```

- `chunk_id` is derived from the file's bytes together with the chunk's position, so the same file uploaded twice produces the same ids. That is what makes a duplicate delivery harmless.
- `headings` and `page_numbers` are what conversion recovered. They are optional — a chunk from a document with no headings simply has an empty list.

Now look at the **Stages** tab. The chunk topic has messages on it and nothing is reading them. The count only goes up. That is the buffering property of a log: the chunks are waiting for a pipeline that does not exist yet, and they will still be there when you deploy one.

YOU NOW HAVE

Chunks from a real document on your own Kafka topic, and a dashboard that shows you them. In the next phase you give them somewhere to go.

TIP

Nothing arrived? Check, in this order: that your subscription shows **Ready** on the application's Event subscriptions tab; that its `chunkstopic` extension matches your topic exactly; and that `./setup.sh topics` actually created it. Section 10.2 has the full list.

6 Phase 4 — The simple enrichment pipeline

Twenty minutes. You deploy a Flink job that reads your chunk topic, enriches every chunk, embeds it and writes the result — you watch it replay everything you uploaded before it existed, and then you search what it indexed and ask it a question.

6.1 Deploy it

```
./pipeline.sh deploy simple
```

The script creates any missing topics, removes anything already deployed for you, waits for its pods to actually go, and then creates a Flink application called `ws-NN-simple`. It prints back what the job will read, write and index. Deploying takes a minute or two while the cluster pulls the image and starts a JobManager and a TaskManager.

> **NOTE**
> Both pipelines are deployed with the OpenSearch sink switched on — the sink is a **list**, `kafka,opensearch`, and the job writes each finished record to both. That is why §5.3 created the index before anything was deployed: a sink whose index is missing fails the job at its first record.

```
./pipeline.sh status
```

Wait until the state is `RUNNING`. In the dashboard's sidebar the pipeline indicator turns green and names the application it found.

> **NOTE**
> Nothing about the job graph is decided by you at deploy time. The pipeline is a container image that was built and published before the workshop; what your command creates is a **FlinkApplication** resource in Confluent Manager for Apache Flink, which tells the cluster to run that image with your topics, your consumer group and your student id.

6.2 What the pipeline does

Phase 4 — the simple pipeline (pipeline.enrich_job)

Stateless: every stage is a function of one chunk. De-duplication arrives in Phase 5.

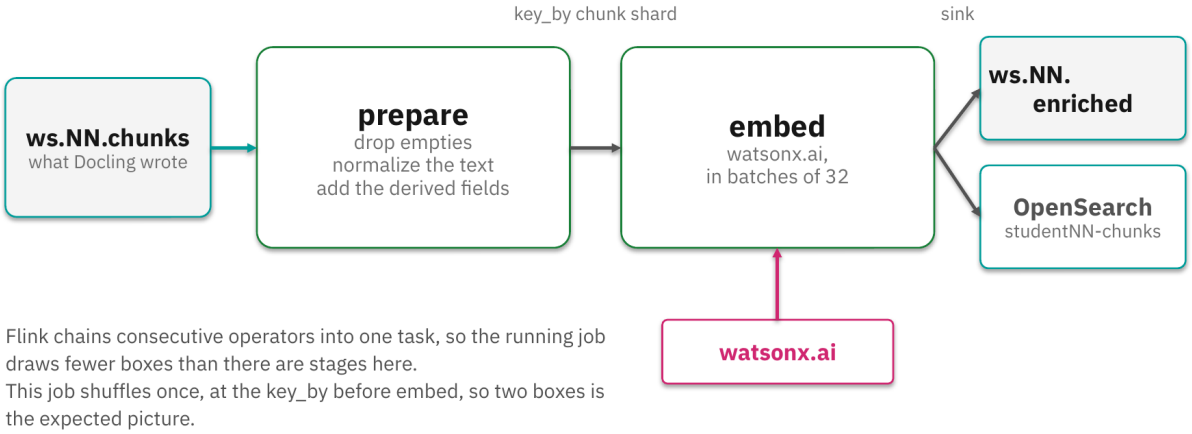


Figure 3. Three stages, one source topic and two destinations, and no state anywhere. Only a key_by breaks a chain, which is why the running job draws fewer boxes than this.

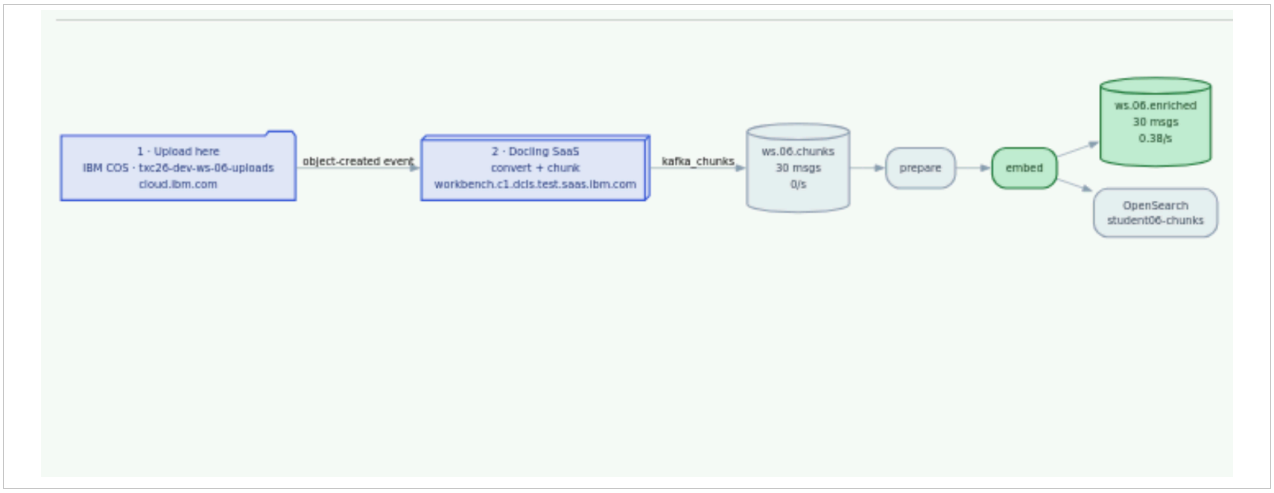
Stage	What it does	Why it is there
prepare	Drops empty and control records, normalizes the text, then adds around twenty derived fields.	Normalizing **before** the text is fingerprinted is what will make the de-duplication you add in Phase 5 insensitive to a soft hyphen or a ligature.
embed	Calls watsonx.ai in batches of 32 and attaches a 768-dimension vector to each chunk.	The one stage that leaves the cluster. Batching turns a thousand round trips into about thirty.
sink	Writes the finished record to `ws.NN.enriched` **and** to your `studentNN-chunks` index.	The topic keeps the pipeline debuggable on its own; the index is what §6.5 searches. `chunk_id` is the document id, so a re-run updates rather than duplicates.

> **NOTE**
>
> Nothing in this pipeline holds state. Every stage is a function of one chunk, and the whole job could — in principle — be a script with a Kafka consumer at one end. That is deliberate: it is the shape worth understanding first, and it is the baseline against which Phase 5 shows you what a stream processor is actually **for**.

6.3 Watch the replay

The job's source starts at the **beginning** of your chunk topic, so the chunks you uploaded in Phase 3 are processed the moment the job starts. You do not have to upload anything to see it work.

1. In the dashboard, open the **Topics** tab. `ws.NN.enriched` climbs from zero towards the number on `ws.NN.chunks`.
2. Open the **Stages** tab and watch the stages colour as they process.

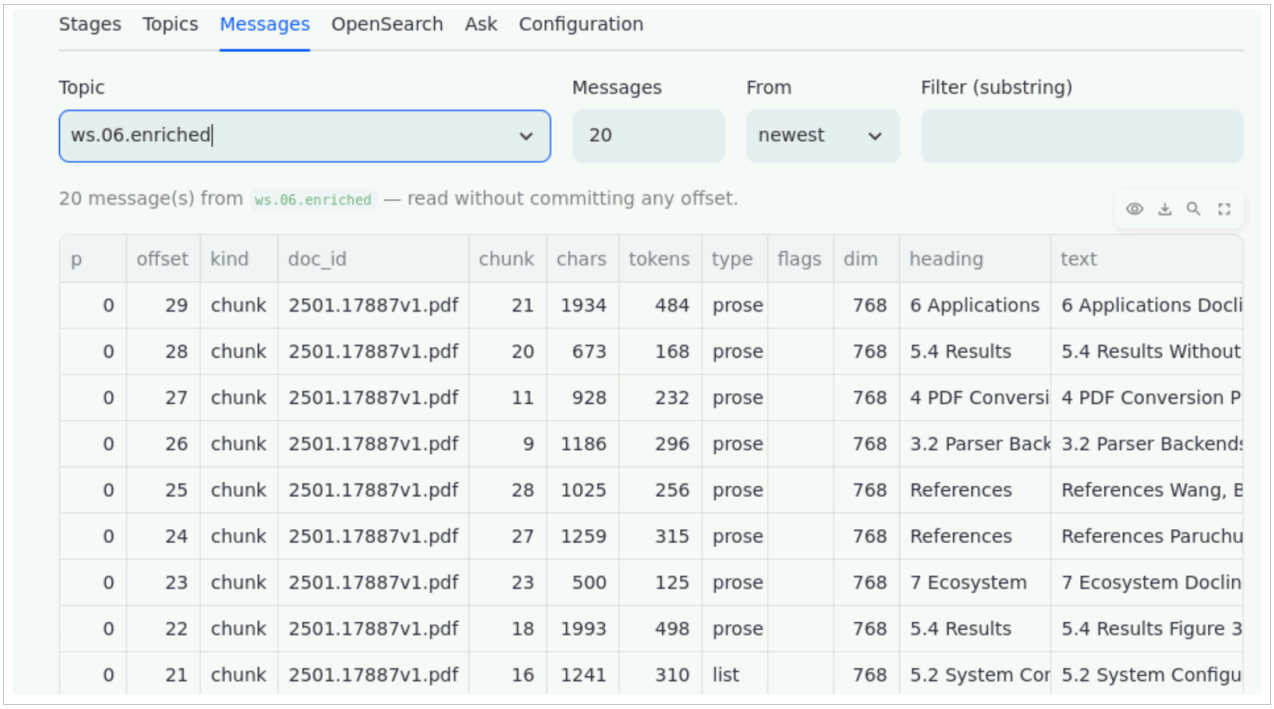


Screen 33. The simple pipeline as the dashboard draws it, with live counts on the topics. The two blue boxes on the left are the upload and Docling — they are not Flink, and they are drawn to make that clear.

> **NOTE**
>
> The Flink UI draws fewer boxes than there are stages. Flink fuses consecutive operators that have the same parallelism and a one-to-one connection into a single task, and PyFlink fuses the Python functions inside such a chain into one operator. Only a `key_by` — a shuffle — breaks a chain. One box per chain is the expected picture, not a sign that a stage is missing.

6.4 Read an enriched record

In the dashboard, open the **Messages** tab, choose `ws.NN.enriched` from the topic list and expand a record.



p	offset	kind	doc_id	chunk	chars	tokens	type	flags	dim	heading	text
0	29	chunk	2501.17887v1.pdf	21	1934	484	prose		768	6 Applications	6 Applications Docli
0	28	chunk	2501.17887v1.pdf	20	673	168	prose		768	5.4 Results	5.4 Results Without
0	27	chunk	2501.17887v1.pdf	11	928	232	prose		768	4 PDF Conversi	4 PDF Conversion P
0	26	chunk	2501.17887v1.pdf	9	1186	296	prose		768	3.2 Parser Back	3.2 Parser Backends
0	25	chunk	2501.17887v1.pdf	28	1025	256	prose		768	References	References Wang, E
0	24	chunk	2501.17887v1.pdf	27	1259	315	prose		768	References	References Paruchu
0	23	chunk	2501.17887v1.pdf	23	500	125	prose		768	7 Ecosystem	7 Ecosystem Doclin
0	22	chunk	2501.17887v1.pdf	18	1993	498	prose		768	5.4 Results	5.4 Results Figure 3
0	21	chunk	2501.17887v1.pdf	16	1241	310	list		768	5.2 System Cor	5.2 System Configu

Screen 34. The same table as in Phase 3, on the enriched topic. `tokens`, `type` and `dim` are filled in now — `dim` is 768, one column's worth of evidence that the embedding actually happened.

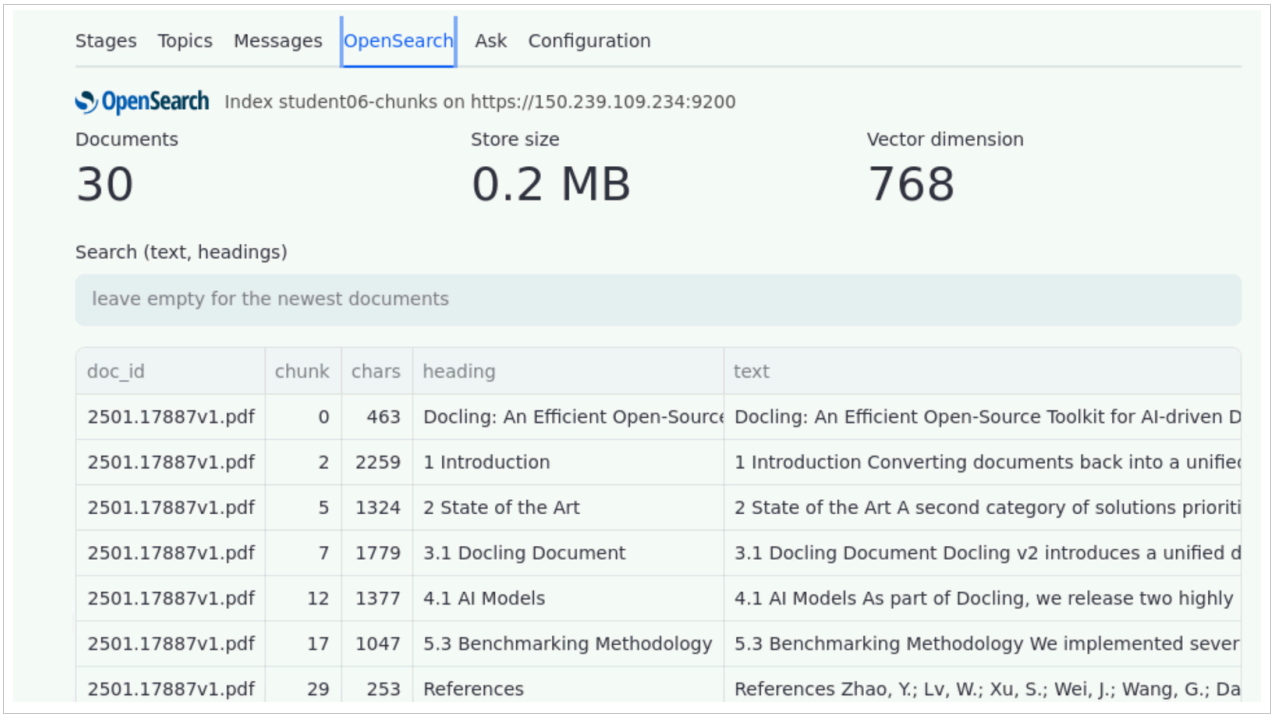
Compare it with the raw chunk you read in Phase 3. Every field that was there is still there, under the same name — the pipeline **adds** and never renames — and beside them you now have:

Group	Fields	What they are for
Size	`char_count` `word_count` `token_count` `avg_word_len`	budgeting a context window at query time; spotting degenerate chunks
Identity	`fingerprint`	a hash of the normalized text — computed here, used as the de-duplication key in Phase 5
Structure	`heading_path` `section_depth` `page_start` `page_end`	citations, and filters scoped to a section
Content	`has_table` `has_list` `has_code` `has_formula`	routing or boosting by chunk kind — tables for numeric questions
Quality	`alpha_ratio` `digit_ratio` `symbol_ratio` `quality_flags` `keep`	dropping OCR noise, page furniture and near-empty fragments
Language	`script`	a coarse writing-system bucket for a multilingual model
Provenance	`ingested_at`	freshness filters and re-index windows
Vector	`embedding`	768 floating-point numbers from watsonx.ai

> **TIP**
> If you would rather have the whole record as raw JSON than as a table, the same thing is one command in a terminal: `uv run scripts/drain_topic.py --topic ws.NN.enriched --full | head -40`.

6.5 Search what you just indexed

The same records went to your OpenSearch index, so there is something to search already — one document and one pipeline is enough. Open the **OpenSearch** tab.



Screen 35. Your index, with the document count, the store size and the vector dimension it was created with. The search box below them is a plain text search over `text` and `headings`.

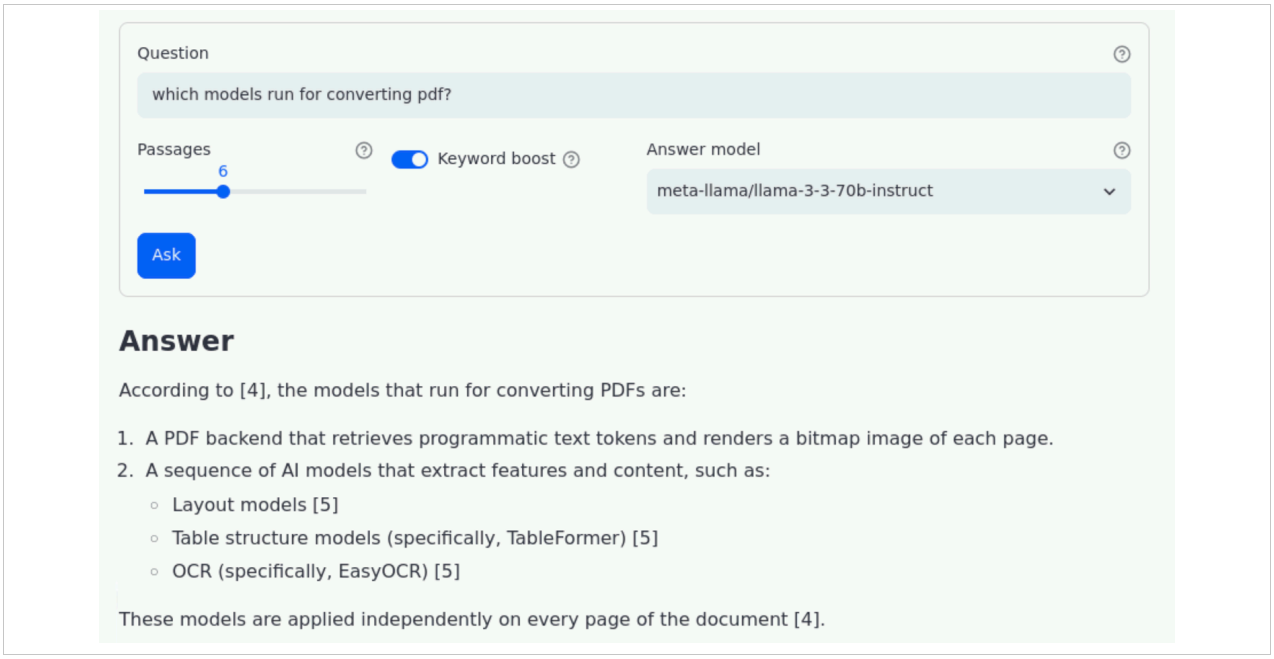
1. Check that **Documents** is climbing towards the number on `ws.NN.enriched`. If it is stuck at zero while the topic fills, the sink is the thing to look at — the **Configuration** tab says which host and index it resolved.
2. Search for a word you know is in the technical note: `bearing`, `HX-420` or `BRG-6207-ZZ`. The rows come back scored, so the ordering is the search's opinion of relevance.
3. Expand a row. It is the whole indexed record, `embedding` included.

> **NOTE**
>
> This is keyword search — no vector involved. The vectors are in the index too, and the next section is what uses them. Seeing the plain search work first is worth the thirty seconds: it separates *did the pipeline index anything* from *is retrieval any good*, which are two different questions and fail for different reasons.

6.6 Ask your own corpus a question

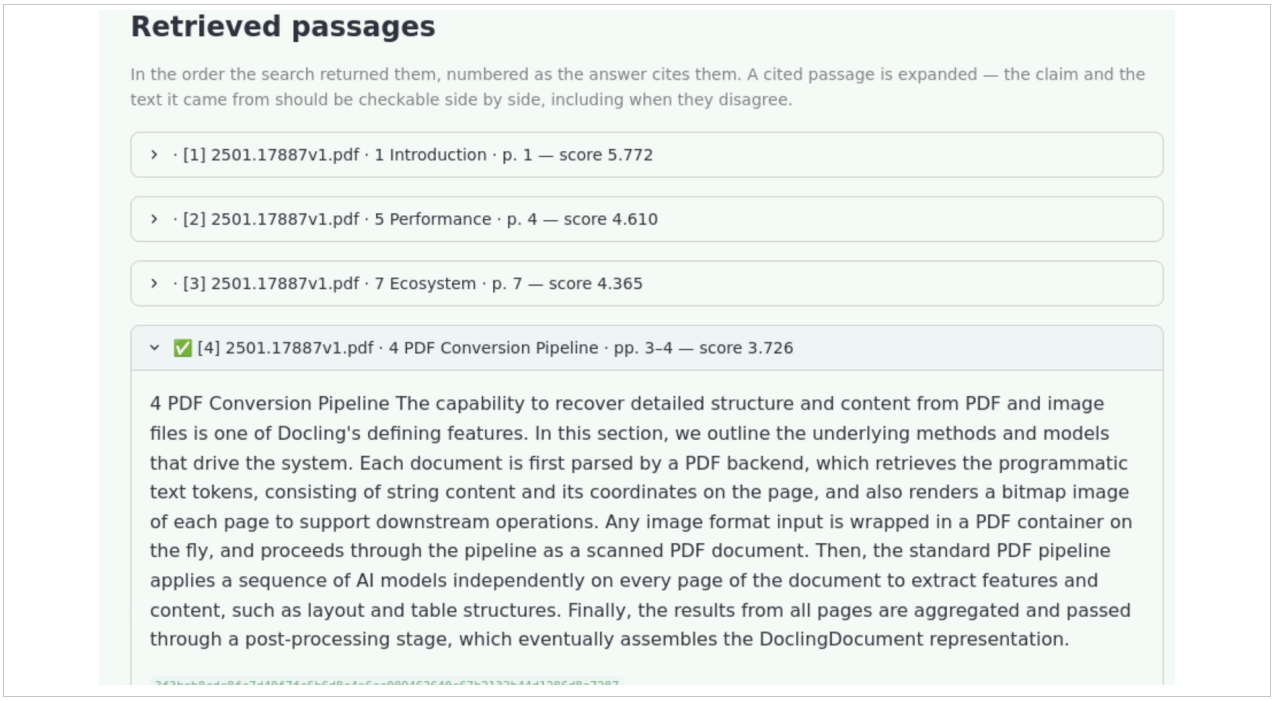
The vectors are in the index too, and with them everything a retrieval-augmented answer needs: the chunks are Docling's, the embeddings are the embed stage's. What is left — embed the question, search, prompt, answer — is four calls, and the **Ask** tab makes them.

1. Open the **Ask** tab and type a question about the technical note — the first one in the table below is a good start.
2. Leave **Passages** at 6 and **Keyword boost** on for the first try.
3. Ask. The answer stays on the page until you ask the next question, whatever the auto-refresh does.



Screen 36. A question, and an answer built only out of your own indexed chunks. The bracketed numbers are citations into the passages below.

Then read the answer against what it was given. Every retrieved passage is listed underneath, in the order the search returned them and numbered as the answer cites them, with the cited ones expanded.



Screen 37. The passages the model was handed, scored. Passage [4] is the one the answer cited, and its text is there to be checked against the claim.

So far your index holds one document, the technical note, and these questions all have their answer in it. Each one tests something different, and the answer the note supports is beside it, so you can tell a right answer from a plausible one:

Ask	The note says	What it tests
Is an HX-420 with serial number HX4-25-0100 affected?	yes: it is inside HX4-24-0655 to HX4-26-0398 (§6)	not a lookup: the model has to place the serial inside a range
Why does my pump overheat in summer?	the shielded bearing BRG-6207-ZZ loses its grease at high ambient temperature (§3)	*overheat* is not in the note: the vector search finds the meaning anyway
What is the revised warning threshold for the HX-420?	75 °C (Table 1)	a value read out of one table row
How often should I re-grease an HX-420 that still has the shielded bearing, and with how much grease?	every 1,500 h, with 8 g (Table 2)	two columns of a table whose header spans two rows
Which part replaces BRG-6207-ZZ, and from which serial number is it fitted in production?	BRG-6207-2RS, from HX4-26-0410 (§5.1)	exact part numbers; ask again with **Keyword boost** off
Who wrote technical note TN-2026-014, and when was it issued?	Ingrid Moe, 12 March 2026	the key-facts panel on the first page

Ask	The note says	What it tests
What is the impeller diameter of the HX-460?	nothing: it is not in the note	a refusal. Read it to the end: a refusal that adds a confident "fact" around it is a hallucination too

NOTE

Keyword boost adds the question's own words back as a text match alongside the vector search. Turn it off and ask something with an exact term in it — a part number such as `BRG-6207-ZZ`, a table caption, a proper noun — and watch the retrieval get worse. Nearest-neighbour search is weak precisely where a term embeds into nothing in particular, which is why production retrieval is almost always hybrid.

TIP

Ask something the documents do **not** answer. A well-behaved prompt says it cannot find it rather than inventing something plausible — and the passage list underneath is how you tell those two apart at a glance.

6.7 Upload another document

UPLOAD NOW

`lab-documents/04-maintenance-manual.pdf`

Phase 4, a second document to ask questions of. Upload it the same way, in the private window.

Then watch the whole chain move at once: the chunk topic's rate rises, the stages go busy, the enriched topic follows a few seconds behind, and the index's document count follows that. This is the loop you will use for the rest of the lab.

Then go back to the **Ask** tab with a question both documents answer: *How often should the HX-420 drive-end bearing be re-greased?* The manual says every 4,000 hours. The technical note, which is newer, shortens that to 2,500 hours with the sealed bearing and 1,500 with the old one. A good answer cites both and says which one applies. A weaker one quotes whichever passage scored highest, and the passage list shows you which happened.

YOU NOW HAVE

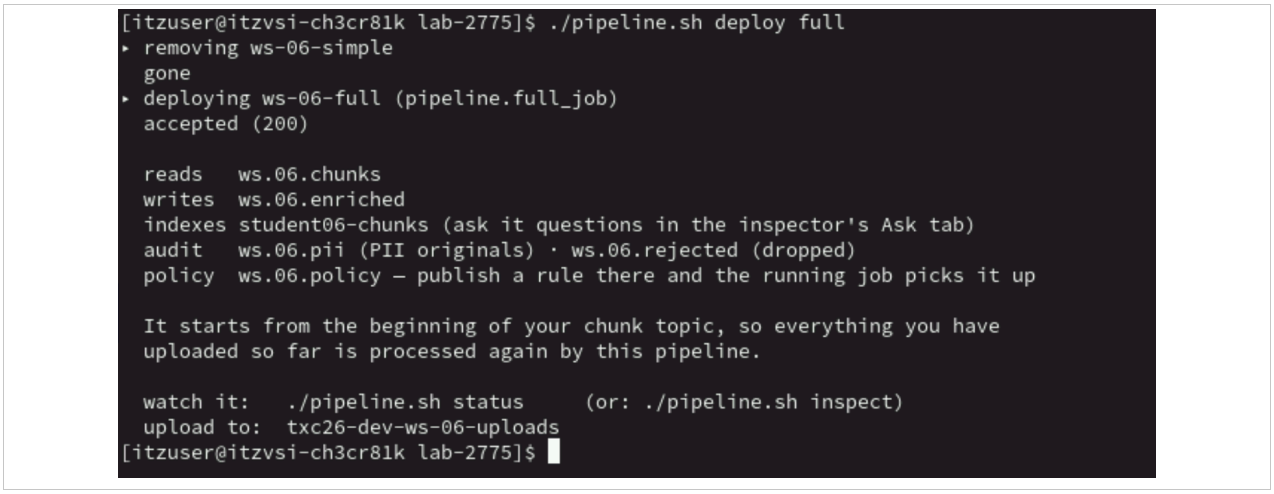
A running Flink job of your own, enriched and embedded records on `ws.NN.enriched`, an index of them you have searched, and an answer out of it with citations. The next phase adds the part that makes all of it fit for a real corpus.

7 Phase 5 — The full pipeline

Twenty minutes. You switch to a pipeline with a control plane: a second source that carries rules, a guard that detects and redacts personal data, a stage that remembers what it has already seen, and two side outputs that keep what the main stream must not carry. Then you change its behaviour without redeploying it.

7.1 Switch pipelines

```
./pipeline.sh deploy full
```



Screen 38. One command: the old application is removed, the new one is accepted, and what it reads, writes, indexes and audits is printed back.

> **WATCH OUT**
>
> `deploy` is a **switch, not an addition**. Your simple pipeline is deleted first and the script waits for its pods to go before creating the new one. That is deliberate: the cluster is shared, and two applications for one student is two TaskManagers holding memory for somebody who can only look at one of them.

Nothing is lost by the switch. The new job starts from the beginning of your chunk topic too, so every chunk you have uploaded today is processed again — this time by a pipeline that has more to say about it. Both pipelines write `ws.NN.enriched`, so the newer records land after the older ones on the same topic and the difference is visible by reading it.

> **NOTE**
> The new job starts with empty state. Everything replayed from the beginning of your chunk topic is seen by the de-duplication stage for the first time, so a full replay is **not** thrown away — but a chunk that appears twice within the replay is.

7.2 What is new

Phase 5 — the full pipeline (pipeline.full_job)

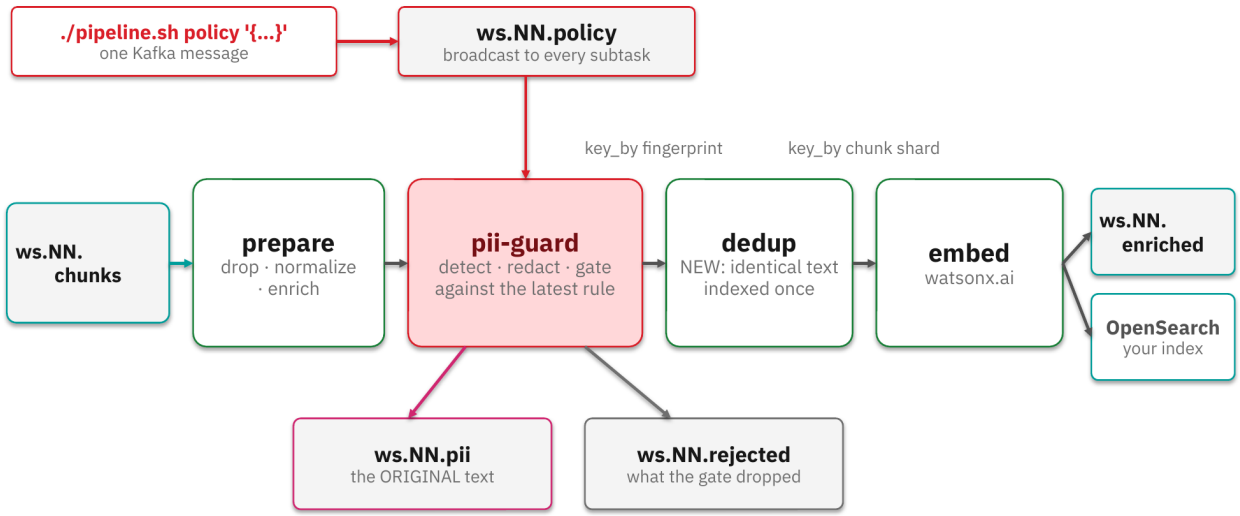
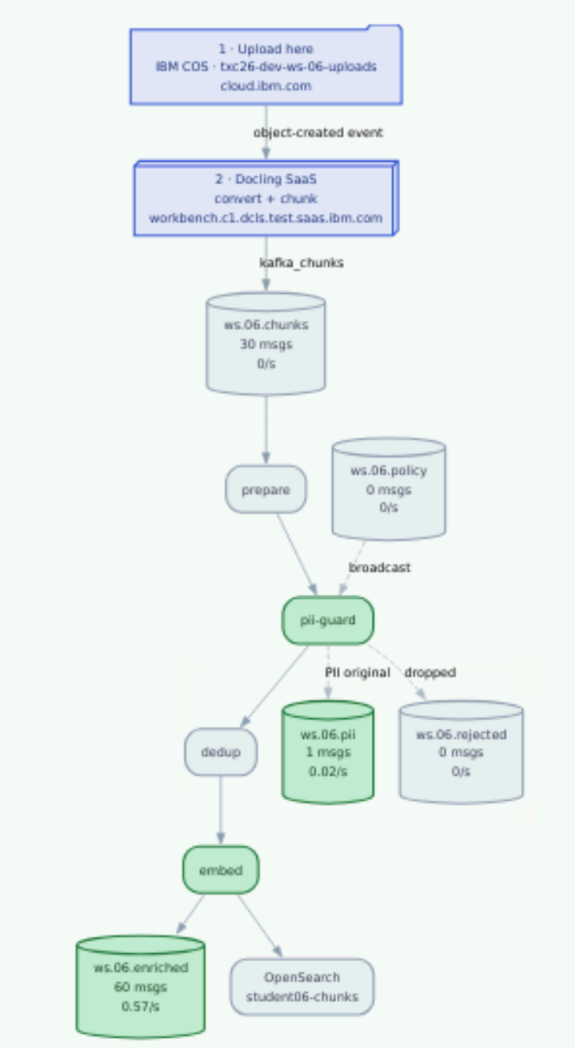


Figure 4. The same spine and the same two destinations, plus a control plane and two side outputs.

- **A second source.** `ws.NN.policy` is broadcast: every message on it goes to every parallel subtask of the guard, which keeps the latest one in state. It is a stream of rules, not of data.
- **A guard after prepare.** It runs the detectors the current rule enables, replaces what it finds, and decides whether the chunk continues at all.
- **A de-duplication stage.** Keyed, expiring state on the chunk's `fingerprint`: text that has been seen before does not go through a second time. This is the first stage in the lab that remembers anything, and §7.6 makes it visible.
- **Two side outputs.** `ws.NN.pii` gets the chunk's **original** text whenever something was found — the audit trail. `ws.NN.rejected` gets chunks the quality gate or a block rule threw away.



Screen 39. The same pipeline as the dashboard draws it, running. `ws.NN.policy` joins the spine from the side, and the two side outputs hang off the guard.

> **NOTE**
>
> The main stream never carries the original text of a chunk that contained personal data. The quarantine copy is written **before** redaction and only there, so the redacted copy is the only one that reaches the index a user can search.

7.3 Document A — an ordinary document

1. There is nothing to upload here: the technical note was replayed through the new job when it started.
2. In the **Messages** tab, open `ws.NN.enriched` and find one of its chunks from after the switch.
3. Look for `pii_count`. It is `0`, and `pii_redacted` is `false`. The guard looked and found nothing.

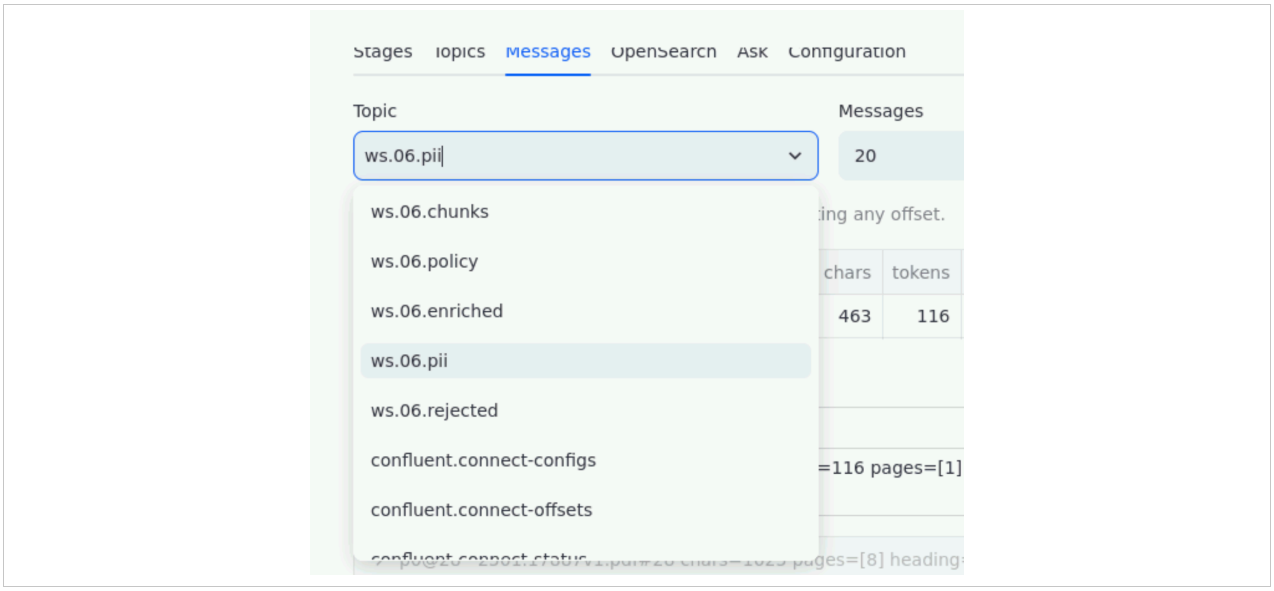
7.4 Document B — a document that contains personal data

UPLOAD NOW

`lab-documents/02-customer-record.pdf`

Phase 5, and only now: the full pipeline's PII guard is running. It contains e-mail addresses, a credit-card number and an IBAN, in ordinary prose.

1. Open `ws.NN.pii` in the **Messages** tab. There is the chunk, with its text exactly as Docling produced it.
2. Now open `ws.NN.enriched` and find the same chunk — same `chunk_id`. Its text has `[REDACTED:EMAIL]` and `[REDACTED:CREDIT_CARD]` where the values were, and it carries `pii_count` and `pii_types`.



Screen 40. All five of your topics are in the Messages tab's list. `ws.NN.pii` is the quarantine copy — the only place the original text of a redacted chunk exists.

Detector	Finds	On by default
`email`	e-mail addresses	yes
`credit_card`	card numbers, confirmed with the Luhn checksum	yes
`iban`	international bank account numbers, confirmed with the ISO 7064 checksum	yes
`phone`	telephone numbers	no
`ip_address`	IPv4 addresses	no

Read that list for what is missing as much as for what is there. Every detector matches a **structured identifier** — something with a rigid shape, and in two cases a checksum that confirms the shape was not a coincidence. None of them looks for a name, a postal address or a date of birth. Those need a model that understands the text, which is a different cost and a different stage; the customer record you just uploaded still has the plant manager's name, *Andrea Keller*, in clear on the enriched topic.

> **TIP**
> Ask the **Ask** tab *How can I contact the plant manager at Brauerei Sonnenberg?* The passage it retrieves says `[REDACTED:EMAIL]`. Redaction happened before indexing, so retrieval never had the address to give away. The mobile number, though, is still there, which is what §7.7 changes.

7.5 The three guards

The stage is one operator, but three separate decisions happen inside it, in order, and each one can send a chunk somewhere other than the index. All three are driven by the same rule from the policy topic.

	What it does	Where the chunk goes
Block list	`blocked_doc_ids` is matched against the chunk's `doc_id` before its text is even read — the escape hatch for a document that should never have been in the bucket	straight to `ws.NN.rejected`, marked `blocked_doc_id`
PII guard	the enabled detectors run over the text; if anything matches, the chunk gains `pii_count` and `pii_types`, and the values are replaced with `[REDACTED:…]` placeholders	a copy of the **original** to `ws.NN.pii`; the redacted copy continues
Quality gate	the text is measured against `min_chars`, together with the `keep` flag that the prepare stage already set from the alphabetic and symbol ratios	`ws.NN.rejected` if `drop_low_quality` is on; otherwise it is flagged with `too_short` and continues

> **NOTE**
> The order matters in one place in particular. The quarantine copy is written **before** the text is replaced, and only to `ws.NN.pii` — so the original exists in exactly one topic, the one with the restricted access, and the redacted copy is the only version that reaches an index a user can search. That is the whole reason this stage is a process function with side outputs rather than a `map`.

Setting `pii_redact` to `false` breaks that arrangement deliberately, and it is worth trying once so you can see the difference: the detectors still run, the counts still land on the record and the quarantine copy is still written, but the main stream keeps the original text. Reporting without removing — which is what you would run first against a corpus you do not know yet, to find out how much there is before deciding what to do about it.

WATCH OUT

This is a demonstrator, not a production PII guard. It is here to make the mechanics concrete — a side output, an audit trail, a rule you can change while the job runs — with something recognisable flowing through them. It is not de-identification, and a corpus that has been through it has not been cleared for anything.

Two limits to keep in mind. It finds **structured identifiers only**: names, addresses, dates of birth, passport numbers and most national identifiers go through untouched. And structure is not meaning — the Luhn checksum rejects about nine in ten arbitrary runs of digits, not all of them, so a row of figures lifted out of a table can be redacted as a card number that never existed. `phone` and `ip_address` ship switched off for the same reason: a numeric table row and a version number look exactly like them.

The part of this that survives contact with a real corpus is the **shape**, not the detectors. Swap them for an entity model behind an asynchronous stage and everything around them — the broadcast rule, the quarantine topic, the counts on the record — is unchanged.

7.6 Document C — the same content, in a different file



UPLOAD NOW

`lab-documents/03-technical-note-reissued.pdf`

Phase 5. A different file, with a different name, page size and typeface, laid out plainly where document A has the house design, and containing the same text. Open both side by side if you like: they do not look alike.

1. Watch `ws.NN.chunks` in the **Topics** tab: it grows by the full number of chunks. Docling converted it and produced every one of them.
2. Watch `ws.NN.enriched`: it does not move.

The de-duplication stage keys on `fingerprint`, a hash of the **normalized text** — not on the file name, not on the file's bytes, and not on `chunk_id`. For that to work, Docling has to recover the same text from both layouts: the same reading order out of two layouts, and the same table out of two renderings of it.

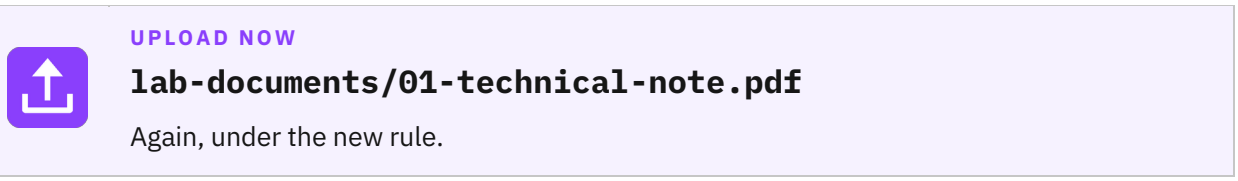
Two different files that say the same thing collapse into one indexed copy, which is what you want for a corpus that has the same policy document filed in four places.

> **NOTE**
> The state has a time-to-live, 24 hours by default. It is a de-duplication window, not a permanent record of everything ever seen — which is what keeps a long-running job's state from growing without bound.

7.7 Change the rules while it runs

This is the point of the phase. Publish one message to the policy topic and the running job behaves differently on its very next record. Nothing is restarted, nothing is rebuilt, nothing is redeployed.

```
./pipeline.sh policy '{"drop_low_quality": true, "min_chars": 200}'
```



1. Open `ws.NN.rejected`. The note's last section, a one-line revision history of 58 characters, is there with `rejected_reason: low_quality`. In a real corpus this is where page furniture, stray captions and section numbers on their own line end up.

Try the other direction, turning on the two detectors that are off by default:

```
./pipeline.sh policy \
  '{"pii_detectors": ["email","credit_card","iban","phone","ip_address"]}'
```



Open `ws.NN.pii`. A chunk that was not there before has arrived: B's *Site and installation* section, whose only identifier is the IP address of the brewery's telemetry gateway. The contacts and billing sections arrive again too, as every chunk with a finding does on every pass. Same job, same process id, different rules.

Now look at `ws.NN.enriched`. It has **not** moved, and that is the de-duplication stage from §7.6 doing its job: the guard runs first, then the dedup stage sees fingerprints it already holds and drops

the chunks. A policy change applies to what flows through next. It does not reach back into what is already indexed. Re-processing a corpus under new rules is a separate decision, a replay with fresh state, which is exactly the kind of operation a log makes possible and a script would have to reinvent.

Key	Type	Does
`pii_detectors`	list of names	which detectors run
`pii_redact`	true / false	redact in the main stream, or only report what was found
`min_chars`	number	the length below which a chunk is considered too small
`drop_low_quality`	true / false	whether a chunk that fails the quality gate is dropped or merely flagged
`blocked_doc_ids`	list of ids	documents to reject outright, by `doc_id`

TIP

A message with an unknown key or a bad value is **rejected and logged**, and the previous rule stays in force — a typo cannot take your job down and cannot silently turn redaction off either. If a change seems to have had no effect, that is the first thing to suspect.

7.8 What you just did

Step back and look at the whole thing. A document arrived because somebody dropped it in a bucket. It was converted and chunked by a service you provisioned in ten minutes. The chunks crossed a broker that any number of other consumers could also read. A stream job cleaned them, threw away what it had already seen, asked a hosted model for vectors, and split what it was not allowed to keep into a separate audit trail — under rules you changed twice while it was running. Somewhere in the middle of that you asked it a question and got an answer back with citations you could check.

None of those pieces knows about the others. That is the property worth taking away: each one can be replaced, scaled or debugged without the rest being told.

YOU NOW HAVE

A pipeline with a control plane, an audit trail, and a demonstration that its behaviour is data rather than deployment.

8 Where to take it next

The pipeline you have is deliberately the smallest thing that is honest about the problem. Three directions to extend it, in rough order of how often they show up in real ingestion:

- **Put a real front end on the retrieval.** The Ask tab is about a hundred lines: embed the question, k-nearest-neighbour with a keyword match beside it, prompt, cite. Swapping either half — a reranker between search and prompt, a different chat model, a conversation instead of one question — changes nothing upstream of the index.
- **Add stages.** Language identification instead of the coarse `script` heuristic; near-duplicate detection with MinHash rather than exact-text de-duplication; keyword or entity extraction stored as a searchable field; a situating sentence prepended to each chunk before embedding.
- **Add consumers.** Nothing about the chunk topic is specific to this pipeline. A second job doing per-document aggregation, or a monitoring stream counting conversions per source, attaches without touching anything that already runs.

The repository has all of it documented, including how the pipeline is built into an image, how the trigger app is deployed, and how the whole class is fanned out across one cluster.

9 Did not finish the lab?

Nothing here disappears when the session ends.

- The repository is public: github.com/IBM/txc2026-docling. Its `README.md` covers everything in this guide and a good deal more, including a local Docker-based stack that needs no workshop cluster at all.
- Your Docling for IBM watsonx trial is yours and keeps working after the lab.
- The self-paced lab room has the same VM images, and your VM gets the same workshop file. Bring your student id — your topics and your bucket are still there.

If you want to finish the exercise on your own machine later, the local stack is the shortest path: it brings up Kafka, Flink and OpenSearch in containers and runs Docling on your laptop, so Phases 3 to 5 work end to end with no cloud accounts.

10 Getting help

10.1 Ask the lab instructor

Raise a hand. The instructors have every student's workshop values, access to the shared account and to the Flink control plane, and can see whether the trigger app received your event — which is the one thing you cannot check yourself.

10.2 Common problems

Symptom	Likely cause	What to do
A lab command is *not found*	this terminal started before the tooling was installed	`export PATH="$HOME/.local/bin:$PATH"`, or open a new terminal
Nothing appears on `ws.NN.chunks` after an upload	the subscription is not ready, or its `chunkstopic` does not match your topic	open the subscription in the Code Engine console and compare the extension with the topic name
Chunks appeared once and then never again	the topic did not exist when the first upload happened, and the conversion is not retried	create the topics, then upload again
The dashboard says nothing is deployed, but you deployed	the dashboard is reading a different student id	check `WS_ID` on the **Configuration** tab
`ws.NN.enriched` stays empty with the job RUNNING	the chunk topic is empty, or the consumer group is shared	check the count on `ws.NN.chunks`; both names come from `WS_ID`, so a wrong id explains both
The OpenSearch tab says the index does not exist	`./setup.sh index` has not been run, or it stopped on a missing password	run it; if the password is missing, ask the instructor
`./setup.sh check` says there is no file for `include: workshop.yaml`	the instructors' file was not on your VM when you ran the bootstrap	ask the instructor, then run the bootstrap from §2.3 again — it leaves your `lab.yaml` alone
The enriched topic fills but the index count stays at zero	the sink reached a different host or index than you are looking at	compare the **Configuration** tab with what `./setup.sh index` printed
The Ask tab says answering needs watsonx.ai	`watsonx.project_id` or the API key is not set for the dashboard	check the **Configuration** tab; retrieval works without them, generation does not
A policy change had no effect	the message was rejected as invalid and the previous rule is still in force	check the JSON quoting; keys are `pii_detectors`, `pii_redact`, `min_chars`, `drop_low_quality`, `blocked_doc_ids`
Deploy fails, or the job never reaches RUNNING	the cluster is full, or the image pull secret expired	`./pipeline.sh status`, then ask the instructor — this one is not yours to fix
Bucket creation refused: name already exists	bucket names are globally unique across IBM Cloud	append your initials and use the new name in the subscription too

Symptom	Likely cause	What to do
The dashboard's bucket link says the bucket does not exist, or the console greets you by your own name	the link opened in an ordinary window, where you are signed in with your IBMid	right-click the link and choose **Open Link in New Private Window**, or copy it into the private window
The IBM Cloud console shows no Object Storage instance	you signed in to a personal account rather than the workshop one	close the private window, open `ibmcloud-signin.html` from your home folder again, and follow its steps — right-click, private window, workshop username

10.3 Useful commands

```
./pipeline.sh status                          # what is deployed, and what it is
doing
./setup.sh topics                             # (re)create your five topics
./setup.sh index                              # (re)create your OpenSearch index
./setup.sh info                               # every name you own — creates nothing
./pipeline.sh deploy simple|full              # switch pipelines
./pipeline.sh policy '{...}'                  # re-rule a running job
./pipeline.sh inspect                         # the dashboard, pointed at your topics
./pipeline.sh delete                          # remove your application

uv run scripts/drain_topic.py --topic ws.NN.chunks --count-only
uv run scripts/drain_topic.py --topic ws.NN.enriched --full

./setup.sh check                              # can this VM reach everything?
```

11 Reference material

	Where
This lab's repository	github.com/IBM/txc2026-docling
Docling for IBM watsonx — the product	ibm.com/products/docling
Docling for IBM watsonx — the documentation	developer.dcls.saas.ibm.com
Docling (open source)	docling.ai

	Where
Docling technical report	arxiv.org/abs/2501.17887
watsonx.ai embedding models	ibm.com/docs/watsonx
Confluent Manager for Apache Flink	docs.confluent.io/platform/current/flink
PyFlink DataStream API	nightlies.apache.org/flink
IBM Cloud Object Storage — event notifications	cloud.ibm.com/docs/cloud-object-storage
IBM Cloud Code Engine — event subscriptions	cloud.ibm.com/docs/codeengine
CloudEvents specification	cloudevents.io

Thank you for coming. If you build something with this, we would like to hear about it — the authors' addresses are on the cover.